第7章 树和二叉树

提纲

CONTENTS

7.1 树

7.2 二 叉 树

7.3 二叉树先序、中序和后序遍历

7.4 二叉树的层次遍历

7.5 二 叉 树的构造

7.6 线索二叉树

7.7 哈夫曼树

7.8 二叉树与树、森林之间的转换

7.9 并查集

7.4 二叉树的层次遍历

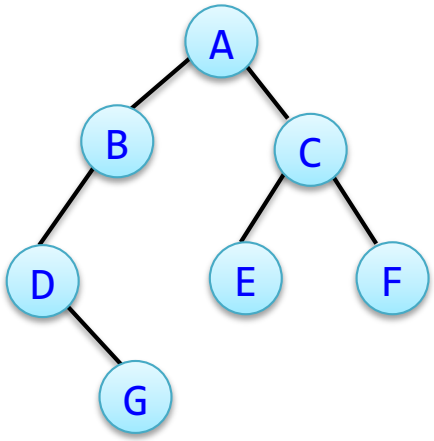
7.4.1 层次遍历过程

若二叉树非空（假设其高度为*h*），则层次遍历的过程如下：

① 访问根结点（第1层）。

② 从左到右访问第2层的所有结点。

③ 从左到右访问第3层的所有结点、…、第*h*层的所有结点。



层次遍历序列为ABCDEFG

7.4.2 层次遍历算法设计

- 在二叉树层次遍历中，对一层的结点访问完后，再按照它们的访问次序对各个结点的左、右孩子顺序访问，这样一层一层进行，先访问的结点其左、右孩子也要先访问，这样与队列的先进先出特点吻合。因此层次遍历算法采用一个队列**qu**来实现。

- 思路：先将根结点***b***进队，在队不空时循环：从队列中出队一个结点***p***，访问它；若它有左孩子结点，将左孩子结点进队；若它有左孩子结点，将左孩子结点进队。如此操作直到队空为止。

```
void LevelOrder(BTree& bt)              //二叉树的层次遍历
{  BTNode* p;
   queue<BTNode*> qu;                   //定义一个队列
   qu.push(bt.r);                       //根结点r进队

   while (!qu.empty())                  //队不空时循环
   {  p=qu.front(); qu.pop();           //出队结点p
      cout << p->data;                  //访问结点p
      if (p->lchild!=NULL)              //有左孩子时将其进队
          qu.push(p->lchild);
      if (p->rchild!=NULL)              //有右孩子时将其进队
          qu.push(p->rchild);
   }
}
```

7.4.3 层次遍历算法的应用

【例7.17】采用层次遍历方法设计例7.14的算法，即求二叉树中第k（$1 \leqslant k \leqslant$二叉树高度）层的结点个数。

解法1

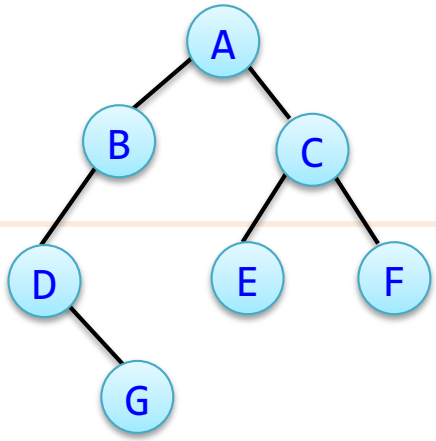
- 用cnt变量计第*k*层结点个数（初始为0）。
- 设计队列中元素类型为QNode类，包含表示当前结点层次lev和结点地址node两个成员变量。先将根结点进队（根结点的层次为1）。在层次遍历中出队一个结点*p*：

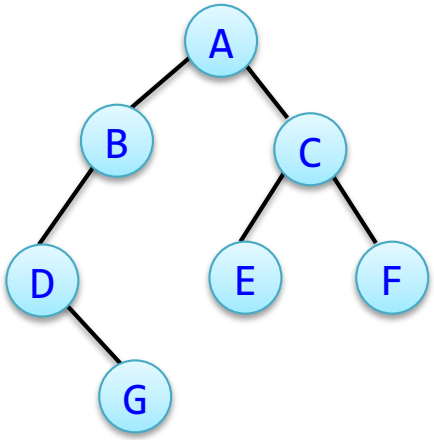
  ① 若结点*p*层次大于*k*，返回cnt（继续层次遍历不可能再找到第*k*层的结点）。

  ② 若结点*p*是第*k*层的结点（p.lev=*k*），cnt增1。

  ③ 若结点*p*的层次小于*k*，将其孩子结点进队，孩子结点的层次为双亲结点的层次加1。

- 最后返回cnt。

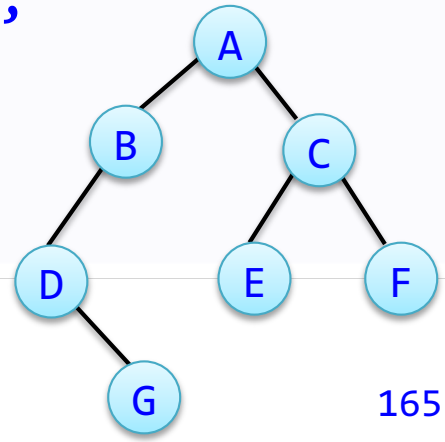


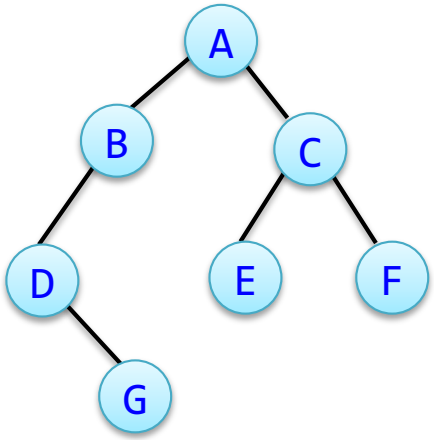


- 用cnt变量计第*k*层结点个数（初始为0）。
- 设计队列中元素类型为QNode类，包含表示当前结点层次lev和结点地址node两个成员变量。先将根结点进队（根结点的层次为1）。在层次遍历中出队一个结点*p*：
  - ① 若结点*p*层次大于*k*，返回cnt（继续层次遍历不可能再找到第*k*层的结点）。
  - ② 若结点*p*是第*k*层的结点（p.lev=*k*），cnt增1。
  - ③ 若结点*p*的层次小于*k*，将其孩子结点进队，孩子结点的层次为双亲结点的层次加1。
- 最后返回cnt。

```
struct QNode                          //队列元素类
{  int lev;                           //结点的层次
   BTNode* node;                      //结点指针
   QNode(int l,BTNode* p)             //构造函数
   {  lev=l;
      node=p;
   }
};
```

```
int KCount1(BTree& bt,int k)          //解法1：求二叉树第k层结点个数
{  int cnt=0;                         //累计第k层结点个数
   queue<QNode> qu;                   //定义一个队列qu
   qu.push(QNode(1,bt.r));            //根结点(层次为1)进队
   while (!qu.empty())                //队不空循环
   {  QNode p=qu.front(); qu.pop();   //出队一个结点
      if (p.lev>k)                    //当前结点的层次大于k，返回cnt
         return cnt;
      if (p.lev==k)
         cnt++;                       //当前结点是第k层的结点,cnt增1
      else                            //当前结点的层次小于k
      {  if (p.node->lchild!=NULL)    //有左孩子时将其进队
            qu.push(QNode(p.lev+1,p.node->lchild));
         if (p.node->rchild!=NULL)    //有右孩子时将其进队
            qu.push(QNode(p.lev+1,p.node->rchild));
      }
   }
   return cnt;
}
```

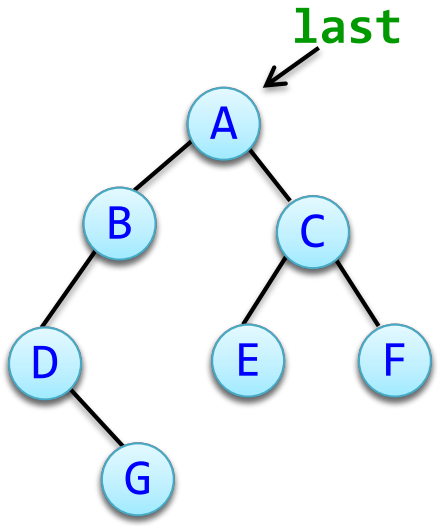





```
int KCount1(BTree& bt,int k)
{  int cnt=0;
   queue<QNode> qu;
   qu.push(QNode(1,bt.r));
   while (!qu.empty())
   {  QNode p=qu.front(); qu.pop();
      if (p.lev>k)
         return cnt;
      if (p.lev==k)
         cnt++;
      else
      {  if (p.node->lchild!=NULL)
            qu.push(QNode(p.lev+1,p.node->lchild));
         if (p.node->rchild!=NULL)
            qu.push(QNode(p.lev+1,p.node->rchild));
      }
   }
   return cnt;
}
```

解法2

层次遍历中某层的最右结点last



last的作用确定一层是否遍历完成！

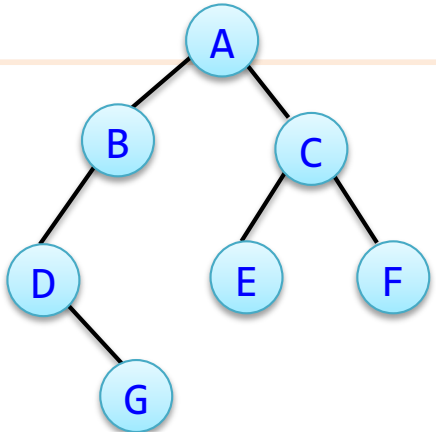
- 用cnt变量计第*k*层结点个数（初始为0）。
- 设计队列仅保存结点引用，置当前层次curl=1，用last变量指示当前层次的最右结点（根结点）进队。将根结点进队，队不空循环：

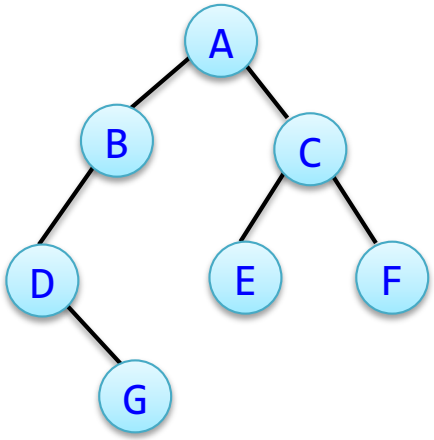
  ① 若curl>*k*，返回cnt（继续层次遍历不可能再找到第*k*层的结点）。

  ② 否则出队结点*p*，若curl=*k*，表示结点*p*是第*k*层的结点，cnt增1。

  ③ 若结点*p*有左孩子*q*，将结点*q*进队，有右孩子*q*，将结点*q*进队（总是用*q*表示进队的结点）。

  ④ 若结点*p*是当前层的最右结点（*p*=last），说明当前层处理完毕，而此时的*q*就是下一层的最右结点，置last=q，curl++进入下一层处理。

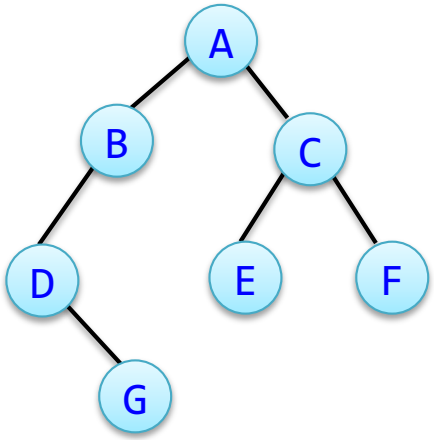




- 用cnt变量计第*k*层结点个数（初始为0）。
- 设计队列仅保存结点引用，置当前层次curl=1，用last变量指示当前层次的最右结点（根结点）进队。将根结点进队，队不空循环：
  - ① 若curl>*k*，返回cnt（继续层次遍历不可能再找到第*k*层的结点）。
  - ② 否则出队结点*p*，若curl=*k*，表示结点*p*是第*k*层的结点，cnt增1。
  - ③ 若结点*p*有左孩子*q*，将结点*q*进队，有右孩子*q*，将结点*q*进队（总是用*q*表示进队的结点）。
  - ④ 若结点*p*是当前层的最右结点（*p*=last），说明当前层处理完毕，而此时的*q*就是下一层的最右结点，置last=q，curl++进入下一层处理。

```
int KCount2(BTree& bt,int k)          //解法2：求二叉树第k层结点个数
{  int cnt=0;                         //累计第k层结点个数
   queue<BTNode* > qu;                //定义一个队列qu
   int curl=1;                        //当前层次,从1开始
   BTNode* last=bt.r,*p,*q;           //第1层最右结点
   qu.push(bt.r);                     //根结点进队
```

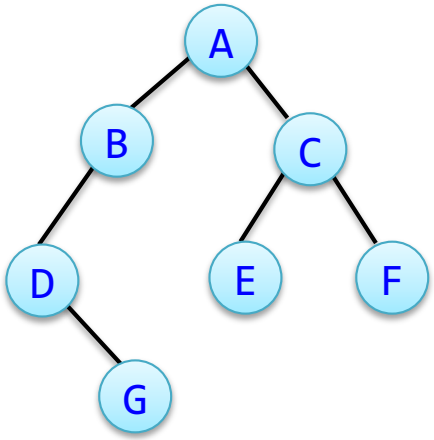
```
  while (!qu.empty())                    //队不空循环
  {  if (curl>k)                         //当层号大于k时返回cnt,不再继续
        return cnt;
     p=qu.front(); qu.pop();             //出队一个结点p
     if (curl==k)
        cnt++;                           //当前结点是第k层的结点,cnt增1
     if (p->lchild!=NULL)                //有左孩子时将其进队
     {  q=p->lchild;
        qu.push(q);
     }
     if (p->rchild!=NULL)                //有右孩子时将其进队
     {  q=p->rchild;
        qu.push(q);
     }
     if (p==last)                        //当前层的所有结点处理完毕
     {  last=q;                          //让last指向下一层的最右结点
        curl++;
     }
  }
  return cnt;
}
```




```
  while (!qu.empty())
  {  if (curl>k)
        return cnt;
     p=qu.front(); qu.pop();
     if (curl==k)
        cnt++;
     if (p->lchild!=NULL)
     {  q=p->lchild;
        qu.push(q);
     }
     if (p->rchild!=NULL)
     {  q=p->rchild;
        qu.push(q);
     }
     if (p==last)
     {  last=q;
        curl++;
     }
  }
  return cnt;
}
```

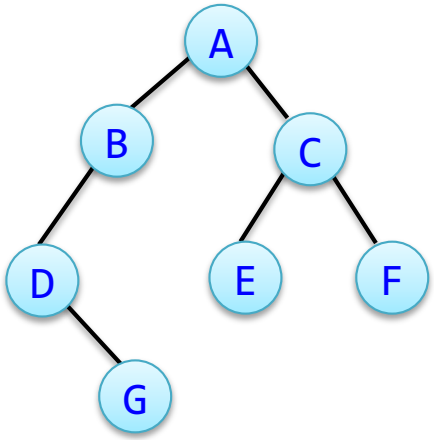
解法3

- 层次遍历是从第一层开始，访问一层的全部结点后（此时该层的全部结点已出队）再访问下一层的结点。上一层遍历完毕，队中恰好是下一层的全部结点。
- 若$k<1$，返回0。
- 否则将根结点进队，当前层次curl=1。队不空循环：
  - ① 若curl=k，队中恰好包含该层的全部结点，直接返回队中元素个数（即第k层结点个数）。
  - ② 否则，求出队中元素个数n（当前层curl的全部结点个数），循环出队n次，每次出队一个结点时将其孩子结点进队。
  - ③ 置curl++，进入下一层处理。
- 最后返回0（$k>$二叉树高度的情况）。



- 层次遍历是从第一层开始，访问一层的全部结点后（此时该层的全部结点已出队）再访问下一层的结点。上一层遍历完毕，队中恰好是下一层的全部结点。
- 若$k<1$，返回0。
- 否则将根结点进队，当前层次curl=1。队不空循环：
  - ① 若curl=k，队中恰好包含该层的全部结点，直接返回队中元素个数（即第k层结点个数）。
  - ② 否则，求出队中元素个数n（当前层curl的全部结点个数），循环出队n次，每次出队一个结点时将其孩子结点进队。
  - ③ 置curl++，进入下一层处理。
- 最后返回0（$k>$二叉树高度的情况）。

```
int KCount3(BTree& bt,int k)          //解法3：求二叉树第k层结点个数
{  if (k<1) return 0;                 //k<1返回0
   queue<BTNode* > qu;                //定义一个队列qu
   int curl=1;                        //当前层次,从1开始
   qu.push(bt.r);                     //根结点进队
   while (!qu.empty())                //队不空循环
   {  if (curl==k)                    //当前层为第k层，返回队中元素个数
         return qu.size();
      int n=qu.size();                //求出当前层结点个数
      for (int i=0;i<n;i++)           //出队当前层的n个结点
      {  BTNode* p=qu.front();qu.pop();     //出队一个结点
         if (p->lchild!=NULL)         //有左孩子时将其进队
            qu.push(p->lchild);
         if (p->rchild!=NULL)         //有右孩子时将其进队
            qu.push(p->rchild);
      }
      curl++;
   }                                  //转向下一层
   return 0;
}
```

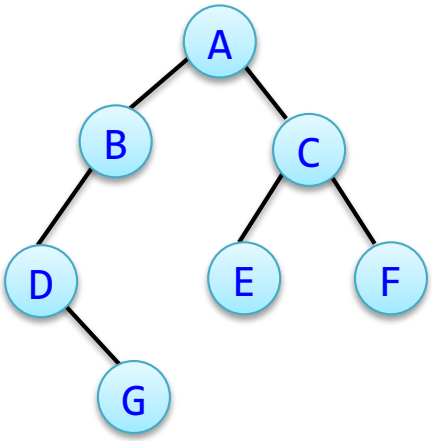


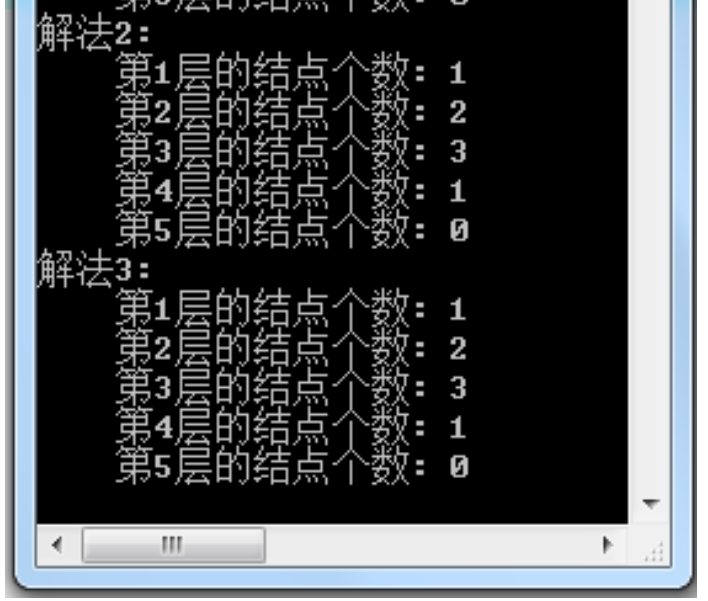

```
int KCount3(BTree& bt,int k)
{  if (k<1) return 0;
   queue<BTNode* > qu;
   int curl=1;
   qu.push(bt.r);
   while (!qu.empty())
   {  if (curl==k)
         return qu.size();
      int n=qu.size();
      for (int i=0;i<n;i++)
      {  BTNode* p=qu.front();qu.pop();
         if (p->lchild!=NULL)
            qu.push(p->lchild);
         if (p->rchild!=NULL)
            qu.push(p->rchild);
      }
      curl++;
   }
   return 0;
}
```

程序验证

```
int main()
{  string str="A(B(D(,G)),C(E,F))";
   BTree bt;
   bt.CreateBTree(str);
   cout << "二叉树bt:"; bt.DispBTree(); cout << endl;
   printf("解法1:\n");
   for (int k=1;k<=5;k++)
      cout << "    第" << k << "层的结点个数: “<<KCount1(bt,k)<<endl;
   printf("解法2:\n");
   for (int k=1;k<=5;k++)
      cout << "    第" << k << "层的结点个数: “<<KCount2(bt,k)<<endl;
   printf("解法3:\n");
   for (int k=1;k<=5;k++)
      cout << "    第" << k << "层的结点个数: “<<KCount3(bt,k)<<endl;
   return 0;
}
```

程序验证


A
B
C
D
E
F
G


F:\C++\C++代码\ch7...
二叉树bt:A(B(D(,G)),C(E,F))
解法1:
第1层的结点个数: 1
第2层的结点个数: 2
第3层的结点个数: 3
第4层的结点个数: 1
第5层的结点个数: 0
解法2:
第1层的结点个数: 1
第2层的结点个数: 2
第3层的结点个数: 3
第4层的结点个数: 1
第5层的结点个数: 0
解法3:
第1层的结点个数: 1
第2层的结点个数: 2
第3层的结点个数: 3
第4层的结点个数: 1
第5层的结点个数: 0

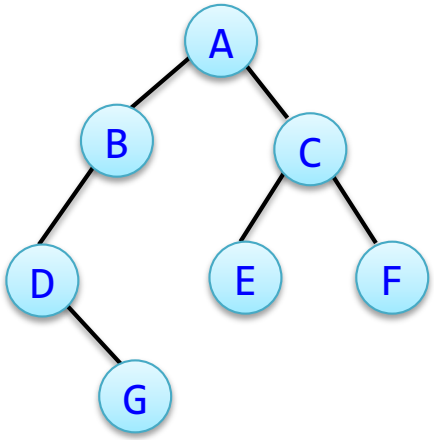
【例7.18】采用层次遍历方法设计例7.15的算法，即输出值为x的结点的所有祖先。

采用例7.17的解法1,设计队列中元素类型为QNode类，包含表示当前结点引用node和双亲pre两个成员。

先将根结点进队（根结点的双亲为NULL）。在层次遍历中出队一个结点p（为队列元素类型而不是二叉树结点类型）：

① 若结点p为x结点（p->node->data=x），从结点p出发通过队列元素回推求出所有祖先结点res（类似用队列求解迷宫路径），返回res。

② 否则，将结点p的孩子结点进队，注意二叉树中孩子结点p->node->lchild和p->node->rchild的双亲结点均为结点p。



采用例7.17的解法1,设计队列中元素类型为QNode类，包含表示当前结点引用node和双亲pre两个成员。

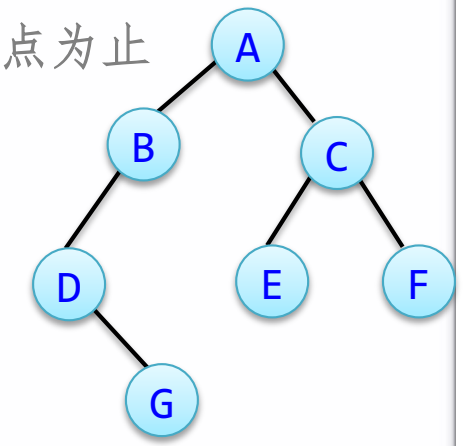
先将根结点进队（根结点的双亲为NULL）。在层次遍历中出队一个结点*p*（为队列元素类型而不是二叉树结点类型）：

① 若结点*p*为*x*结点（p->node->data=*x*），从结点*p*出发通过队列元素回推求出所有祖先结点res（类似用队列求解迷宫路径），返回res。

② 否则，将结点*p*的孩子结点进队，注意二叉树中孩子结点p->node->lchild和p->node->rchild的双亲结点均为结点*p*。

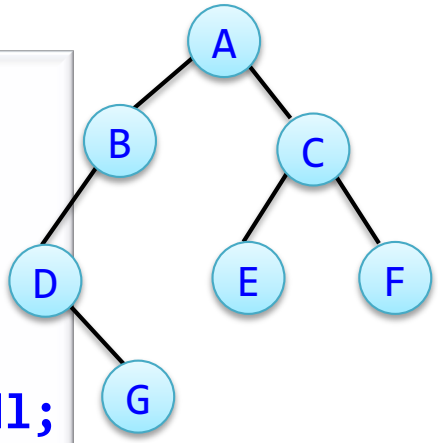
```
struct QNode                                    //QNode类型
{   BTNode* node;                               //当前结点指针
    QNode* pre;                                 //当前结点的双亲结点

    QNode(BTNode* p1,QNode* p2)                 //构造函数
    {   node=p1;
        pre=p2;
    }
};
```

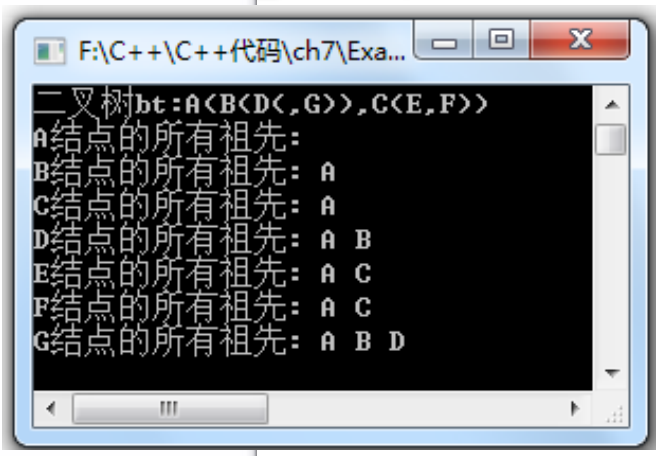
```
void Ancestor4(BTree& bt,char x,vector<char>& res)
//层次遍历求x结点的祖先
{  queue<QNode* > qu;                          //定义一个队列qu
   qu.push(new QNode(bt.r,NULL));              //根结点(双亲为NULL)进队
   while (!qu.empty())                         //队不空循环
   {  QNode* p=qu.front(); qu.pop();           //出队一个结点
      if (p->node->data==x)                    //当前结点为x结点
      {  QNode* q=p->pre;                      //q为双亲
         while (q!=NULL)                       //找到根结点为止
         {  res.push_back(q->node->data);
            q=q->pre;
         }
         reverse(res.begin(),res.end());
         return;
      }
      if (p->node->lchild!=NULL)               //有左孩子时将其进队
         qu.push(new QNode(p->node->lchild,p));    //置其双亲为p
      if (p->node->rchild!=NULL)               //有右孩子时将其进队
         qu.push(new QNode(p->node->rchild,p));    //置其双亲为p
   }
}
```



程序验证

```
int main()
{  string str="A(B(D(,G)),C(E,F))";
   BTree bt;
   bt.CreateBTree(str);
   cout << "二叉树bt:"; bt.DispBTree(); cout << endl;
   for (char ch='A';ch<='G';ch++)
   {  cout <<  ch << "结点的所有祖先:";
      vector<char> res;                    //存放祖先
      Ancestor4(bt,ch,res);
      for (int i=0;i<res.size();i++)
          cout << " " << res[i];
      cout << endl;
   }
   return 0;
}
```



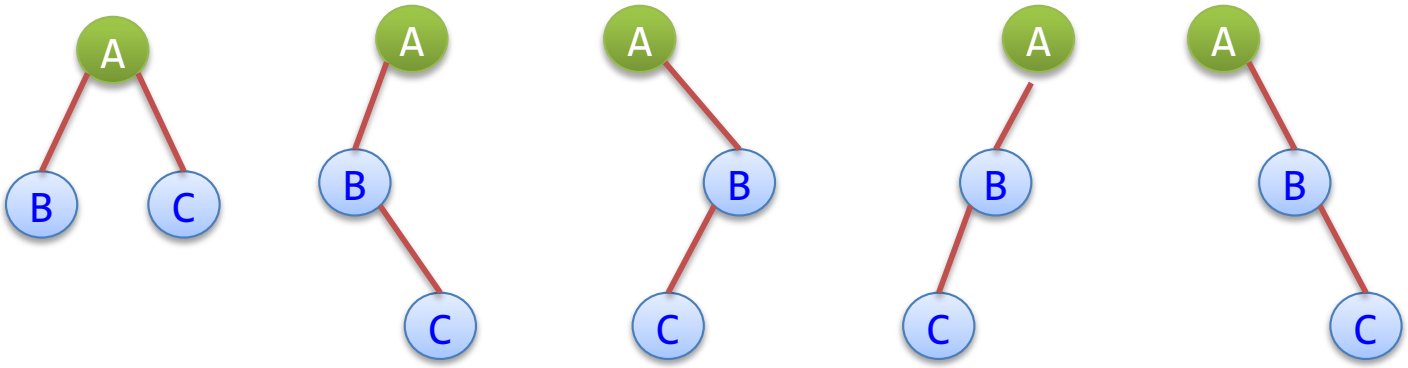


7.5 二叉树的构造

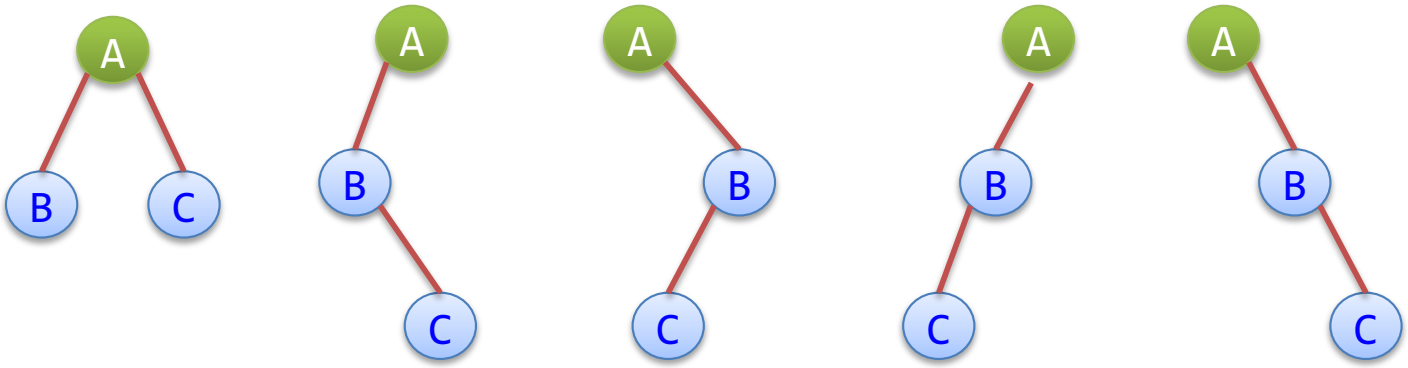
7.5.1 由先序/中序序列或后序/中序序列构造二叉树

- 一棵所有结点值不同的二叉树，其先序、中序、后序和层次遍历都是唯一的，也就是说一棵这样的二叉树，不可以有两种不同的先序遍历序列，也不可能有两种不同的中序序列。
- 二叉树的构造就是给定某些遍历序列，反过来唯一地确定该二叉树。

二叉树 遍历序列	图(a)的二叉树	图(b)的二叉树	图(c)的二叉树	图(d)的二叉树	图(e)的二叉树
先序遍历序列					
中序遍历序列					
后序遍历序列					

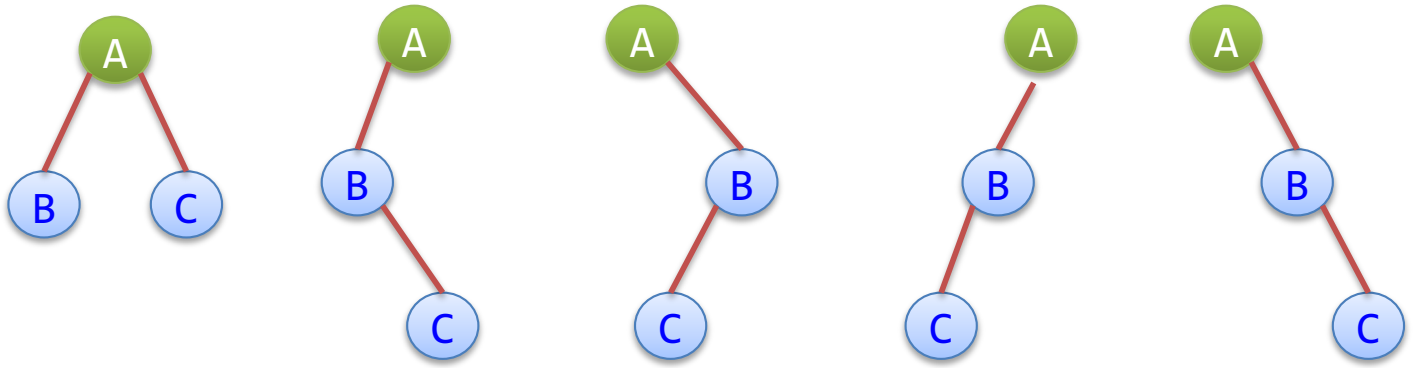


二叉树 遍历序列	图(a)的二叉树	图(b)的二叉树	图(c)的二叉树	图(d)的二叉树	图(e)的二叉树
先序遍历序列	**ABC**	**ABC**	**ABC**	**ABC**	**ABC**
中序遍历序列	**BAC**	**BCA**	**ACB**	**CBA**	**ABC**
后序遍历序列	**BCA**	**CBA**	**CBA**	**CBA**	**CBA**



从中看到，对于不同形态的二叉树：

- 先序遍历序列可能相同（图中5棵二叉树的先序遍历序列均相同）。
- 中序遍历序列可能相同。
- 后序遍历序列可能相同（图(b)～(e)的后序遍历序列均相同）
- 先序遍历序列和后序遍历序列可能都相同（图(d)和(e)的先序遍历序列和后序遍历序列均相同）。



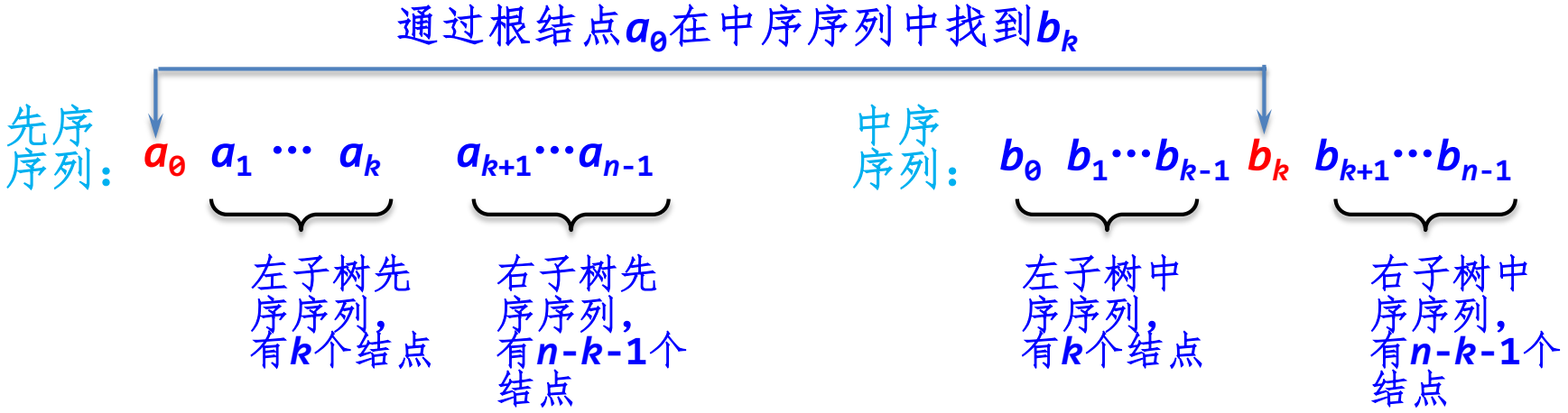
实际上，对于含2个或者以上结点的二叉树，在先序、中序和后序遍历序列中：

- 由先序遍历序列和中序遍历序列能够唯一确定一棵二叉树。
- 由后序遍历序列和中序遍历序列能够唯一确定一棵二叉树。
- 由先序遍历序列和后序遍历序列不能唯一确定一棵二叉树。

定理7.1 任何n（$n\geq 0$）个不同结点的二叉树，都可由它的中序序列b和先序序列a唯一地确定。

证明

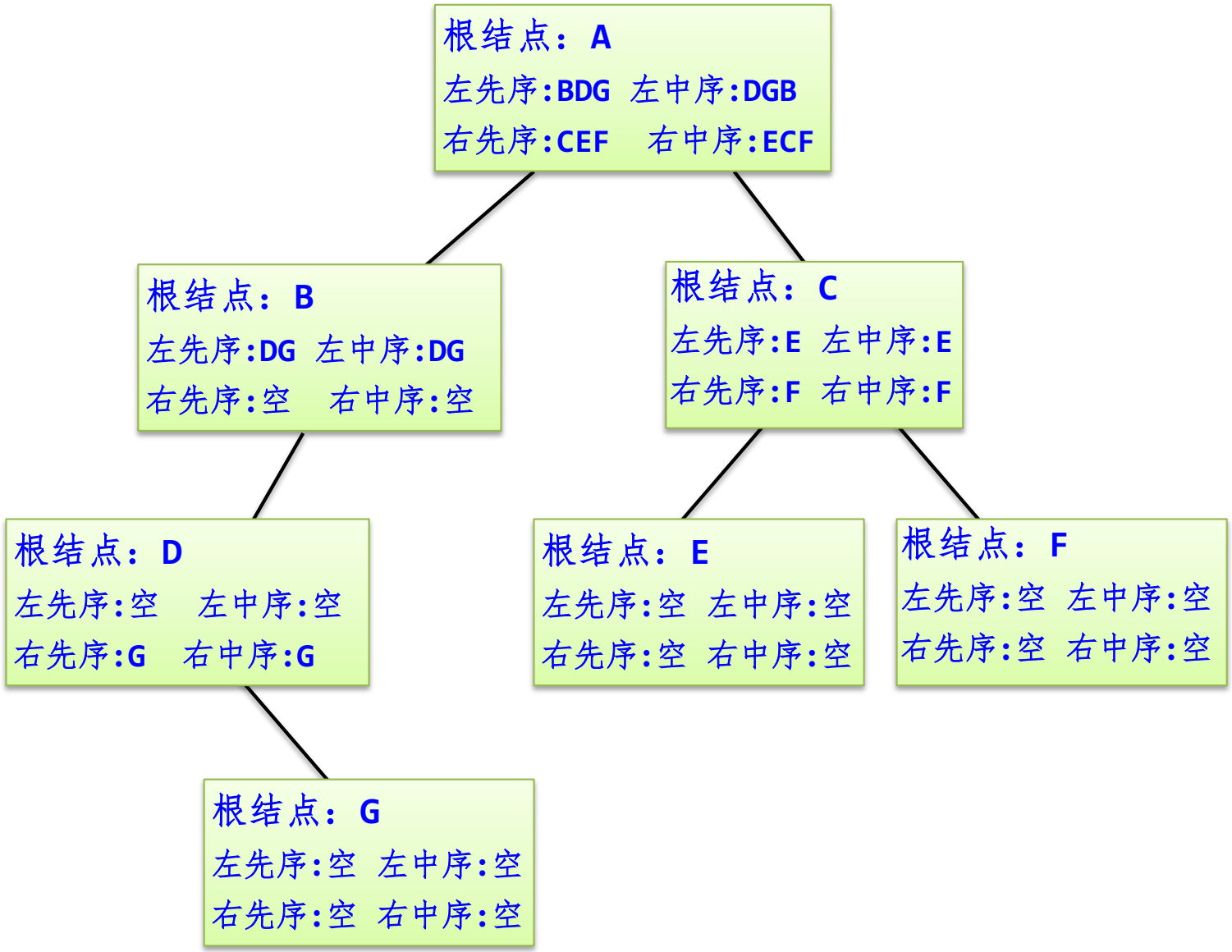
- 由a_0（根结点）找到b_k。



- 若b_k前面有k个结点，则左子树有k个结点，右子树有$n-k-1$个结点。
- 可以求出左右子树的中序序列和先序序列。
- 这样根结点是确定的，左右子树也是确定的，则该二叉树是确定的。

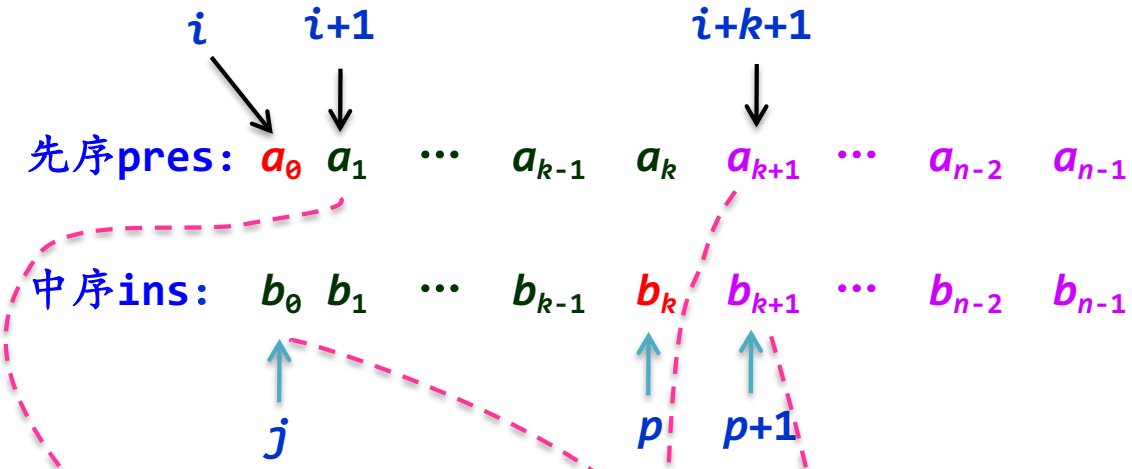
已知先序序列为**ABDGCEF**，中序序列为**DGBAECF**，则构造二叉树的过程：

已知先序序列为ABDGCEF，中序序列为DGBAECF，则构造二叉树的过程：



```
void CreateBTree1(BTree& bt,vector<char> pres,vector<char> ins)
//由先序序列pres和中序序列ins构造二叉链
{  int n=pres.size();
   bt.r=CreateBTree11(pres,0,ins,0,n);
}
```

由先序序列pres[i..i+n-1]和中序序列ins[j..j+n-1]创建二叉链b

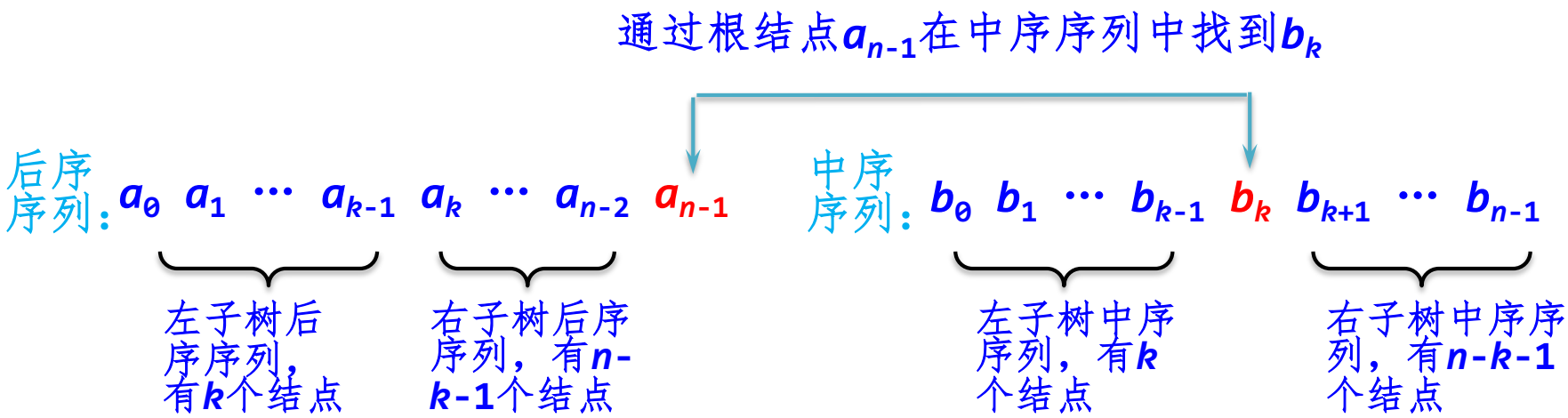



```
BTNode* CreateBTree11(vector<char> pres,int i,
                      vector<char> ins,int j,int n) //被CreateBTree1调用
{  if (n<=0) return NULL;
   char d=pres[i];                                  //取根结点值d
   BTNode* b=new BTNode(d);                         //创建根结点(值为d)
   int p=j;
   while (ins[p]!=d) p++;                           //ins中找根索引p
   int k=p-j;                                       //左子树中结点个数k
   b->lchild=CreateBTree11(pres,i+1,ins,j,k);       //递归构造左子树
   b->rchild=CreateBTree11(pres,i+k+1,ins,p+1,n-k-1); //递归构造右子树
   return b;
}
```

定理7.2 任何n（$n\geq 0$）个不同结点的二叉树，都可由它的中序序列和后序序列唯一地确定。

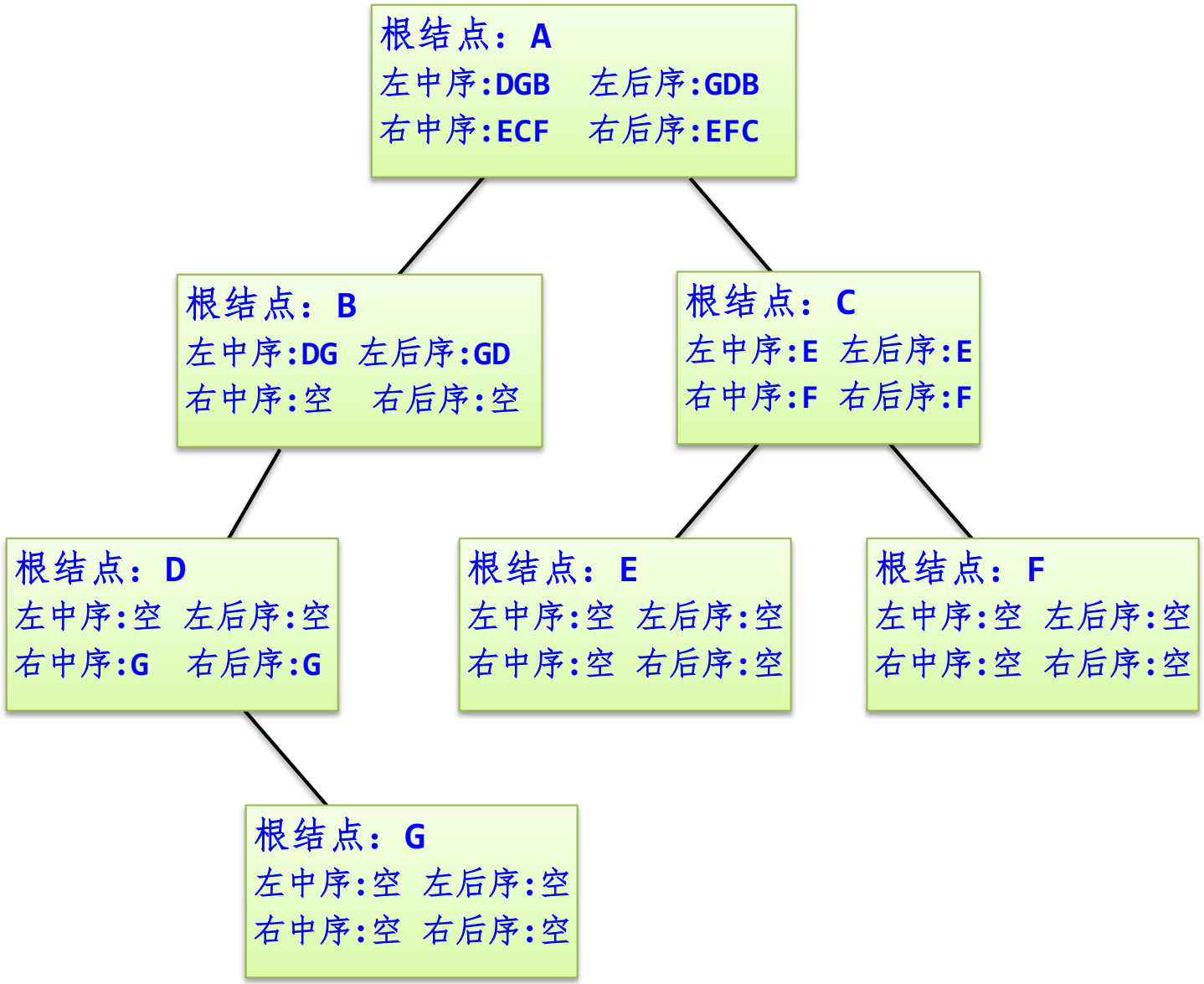
证明

- 由a_{n-1}（根结点）找到b_k。



- 若b_k前面有k个结点，则左子树有k个结点，右子树有$n-k-1$个结点。
- 可以求出左右子树的中序序列和后序序列。
- 这样根结点是确定的，左右子树也是确定的，则该二叉树是确定的。

已知中序序列为DGBAECF，后序序列为GDBEFCA，则构造二叉树的过程



```
void CreateBTree2(BTree& bt,vector<char> posts,vector<char> ins)
//由后序序列posts和中序序列ins构造二叉链
{  int n=posts.size();
   bt.r=CreateBTree21(posts,0,ins,0,n);
}
```

由后序序列posts[*i*..*i*+*n*-1]和中序序列ins[*j*..*j*+*n*-1]创建二叉链*b*

```
BTNode* CreateBTree21(vector<char> posts,int i,
                      vector<char> ins,int j,int n)
//被CreateBTree2调用
{  if (n<=0) return NULL;
   char d=posts[i+n-1];                    //取后序序列尾元素即根值d
   BTNode* b=new BTNode(d);                //创建根结点(结点值为d)
   int p=j;
   while (ins[p]!=d) p++;                  //在ins中找到根结点的索引p
   int k=p-j;                              //确定左子树中结点个数k
   b->lchild=CreateBTree21(posts,i,ins,j,k);       //递归构造左子树
   b->rchild=CreateBTree21(posts,i+k,ins,p+1,n-k-1); //递归构造右子树
   return b;
}
```

【例7.19】若某非空二叉树的先序序列和后序序列正好相同，则该二叉树的形态是什么？

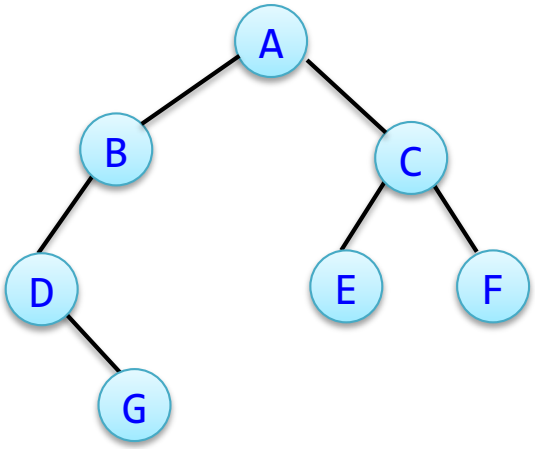
二叉树的先序序列是NLR，后序序列是LRN。

N L R = L R N

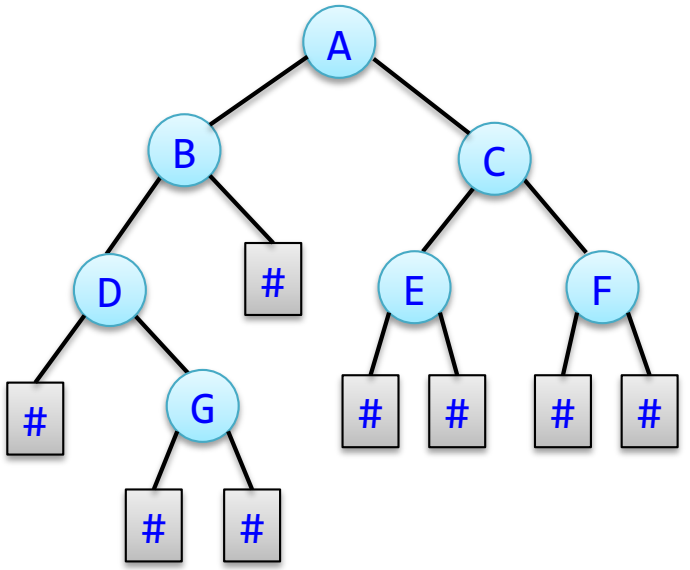
则L和R均为空。所以满足条件的二叉树只有一个根结点。

7.5.2* 序列化和反序列化

仅仅讨论先序遍历序列化和反序列化。



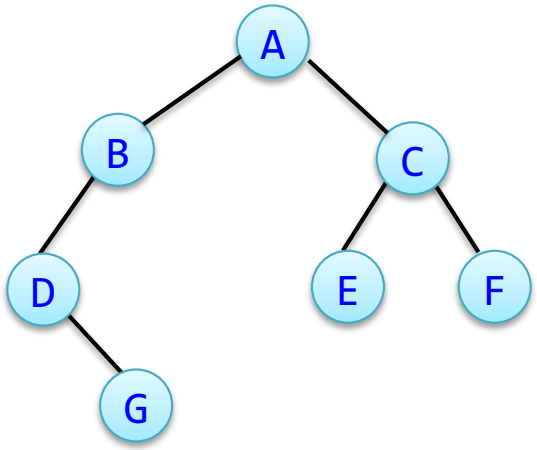
加上外部结点



先序遍历

序列化序列 ⟶ "ABD#G###CE##F##"

```
string PreOrderSeq(BTree& bt)          //二叉树bt的序列化
{
   return PreOrderSeq1(bt.r);
}

string PreOrderSeq1(BTNode* b)         //被PreOrderSeq调用
{  if (b==NULL) return "#";
   string s(1,b->data);                //含根结点
   s+=PreOrderSeq1(b->lchild);         //产生左子树的序列化序列
   s+=PreOrderSeq1(b->rchild);         //产生右子树的序列化序列
   return s;
}
```



序列化

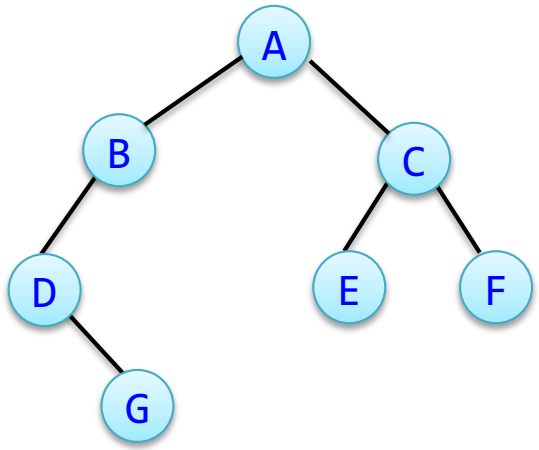
序列化序列

"ABD#G###CE##F##"

序列化序列

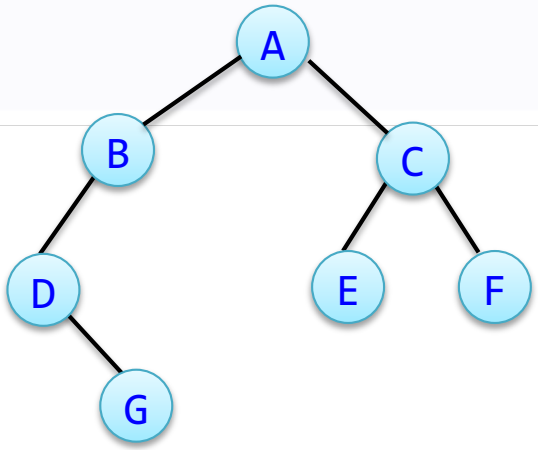
"ABD#G###CE##F##"

反序列化

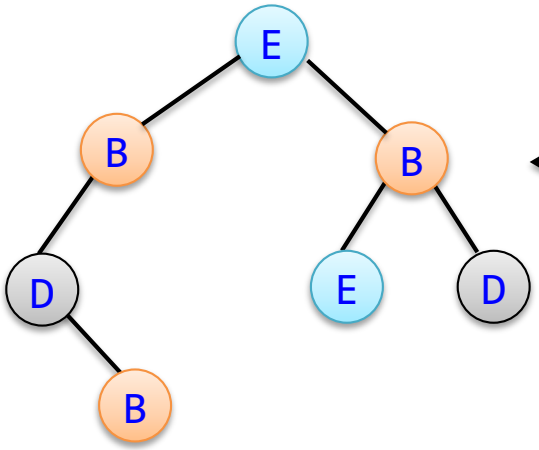


```
void CreateBTree3(BTree& bt,string s)
//由序列化序列s创建二叉链：反序列化
{  int i=0;
   bt.r=CreateBTree31(s,i);
}
BTNode* CreateBTree31(string s,int& i)
//被CreateBTree3调用，其中参数i相当于全局变量
{  if (i>=s.length()) return NULL;          //i超界返回NULL
   char d=s[i]; i++;                        //取s[i]的值d
   if (d=='#') return NULL;                 //若为"#"，返回NULL
   BTNode* b=new BTNode(d);                 //创建根结点(结点值为d)
   b->lchild=CreateBTree31(s,i);            //递归构造左子树
   b->rchild=CreateBTree31(s,i);            //递归构造右子树
   return b;
}
```

"ABD#G###CE##F##"



由于反序列化构造二叉树过程中不像先序/中序和后序/中序那样需要根结点值的比较，所以适合构造结点值相同的二叉树。



← 无法由先序序列和中序序列构造！

先序序列：**EBDBBED**
中序序列：**DBBEEBD**

7.6 线索二叉树

7.6.1 线索二叉树的定义

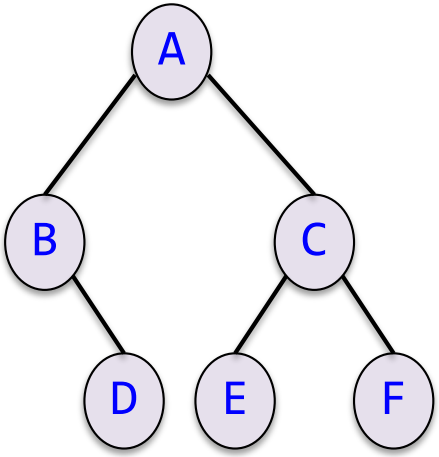
- 对于具有n个结点的二叉树，采用二叉链存储结构时，每个结点有两个指针域，总共有2n个指针域。
- 其中只有n-1个结点被有效指针所指向，即有n-1个非空指针域。
- 所以共有2n-(n-1) = n+1个空链域。

7.6 线索二叉树

7.6.1 线索二叉树的定义

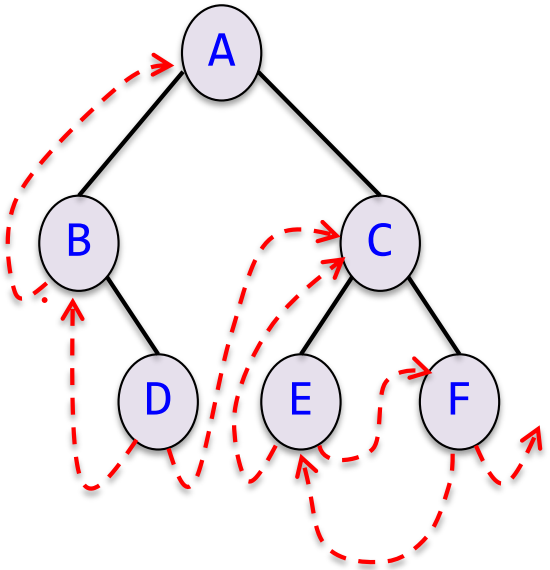
- 利用这些空链域存放在某种遍历次序下该结点的前驱结点和后继结点的指针，这些指针称为线索，加上线索的二叉树称为线索二叉树。
- 线索二叉树分为先序、中序和后序线索二叉树。
- 对二叉树以某种方式遍历使其变为线索二叉树的过程称为线索化。

图中虚线为线索。



二叉树

先序线索二叉树



先序序列：**ABDCEF**

在原二叉链中增加了**ltag**和**rtag**两个标志域。

ltag=
- **0** 表示**lchild**指向结点的左孩子
- **1** 表示**lchild**指向结点的前驱结点即为线索

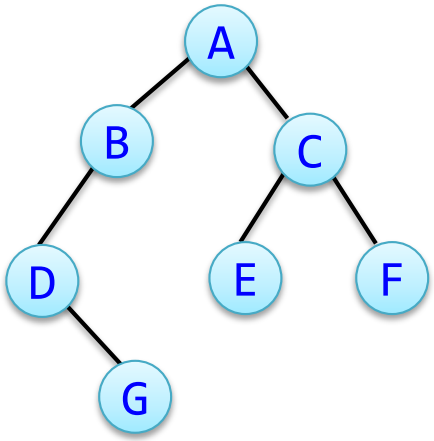
rtag=
- **0** 表示**rchild**指向结点的右孩子
- **1** 表示**rchild**指向结点的后继结点即为线索

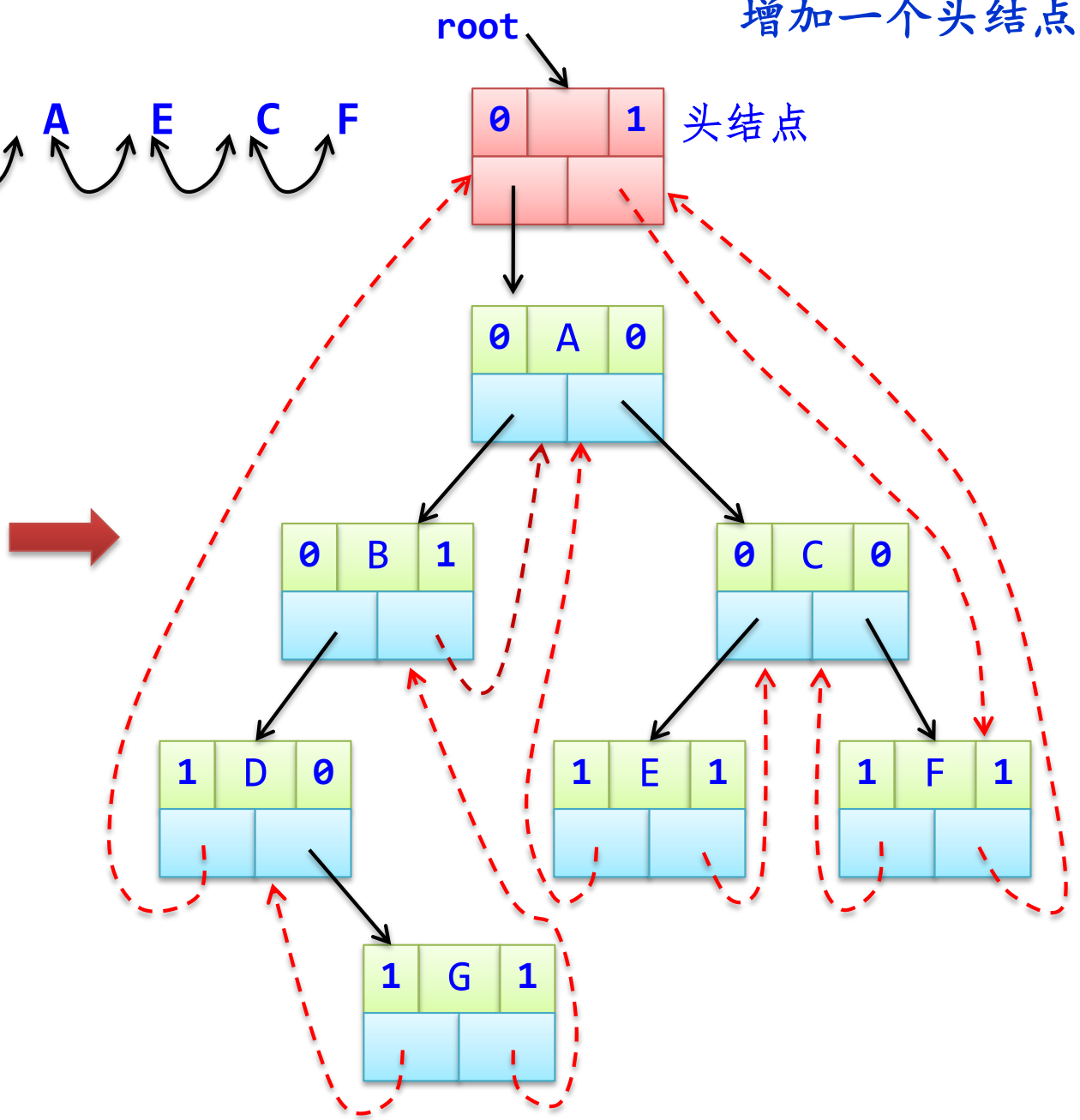
ltag	lchild	data	rchild	rtag

以中序线索二叉树为例

在线索二叉树中
增加一个头结点

中序序列：D G B A E C F

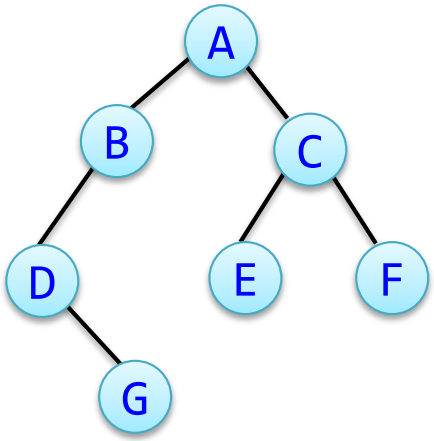


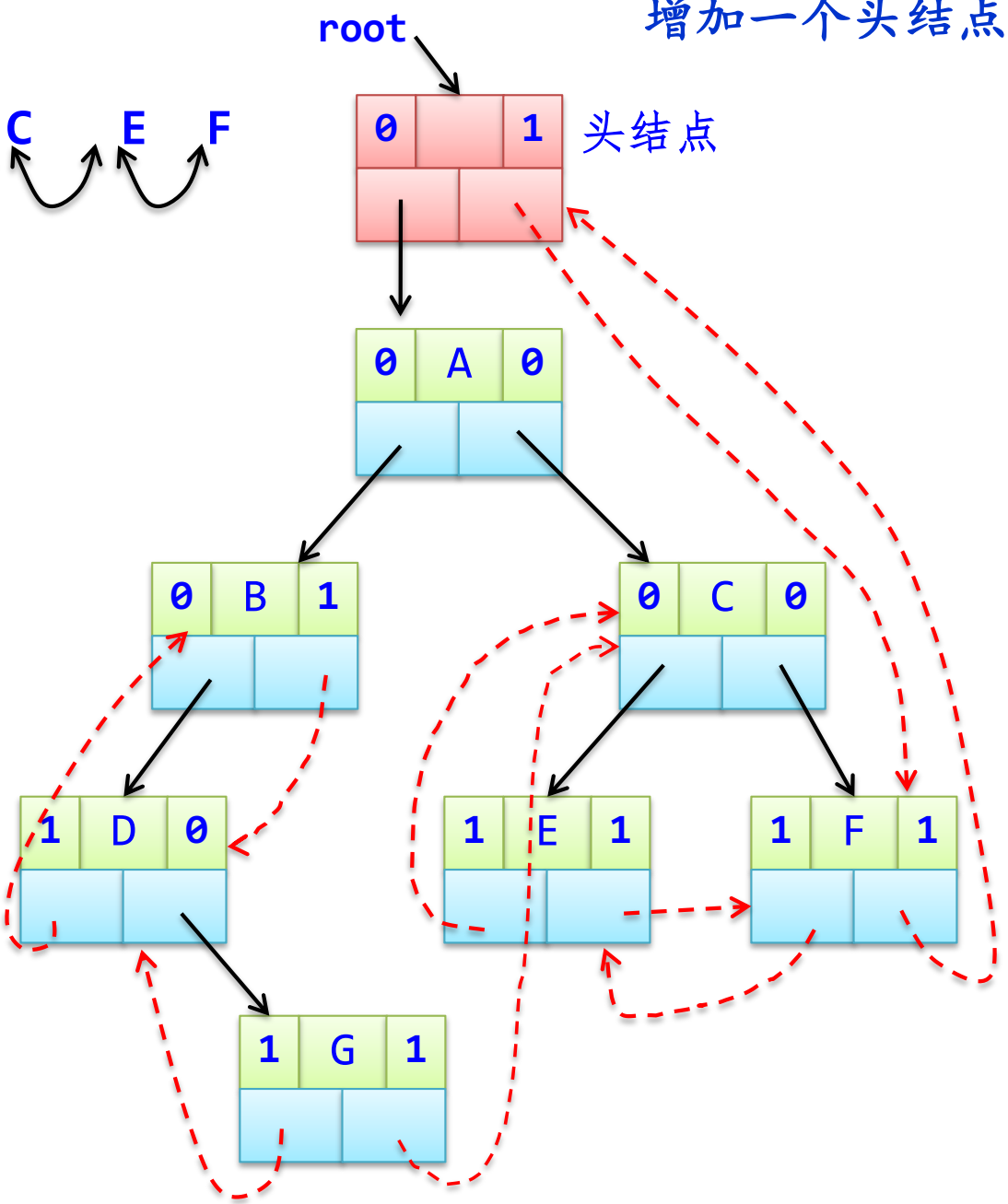


练习：以先序线索二叉树为例

在线索二叉树中
增加一个头结点

先序序列：A B D G C E F





7.6.2 线索化二叉树

建立某种次序的线索二叉树过程：

- 以该遍历方法遍历一棵二叉树。
- 在遍历的过程中，检查当前访问结点的左、右指针域是否为空：
  - 如果左指针域为空，将它改为指向前驱结点的线索；
  - 如果右指针域为空，将它改为指向后继结点的线索。

以中序线索二叉树为例，讨论建立线索二叉树的算法。

建立中序线索二叉树的算法

- CreatThread(b)算法：对以二叉链存储的二叉树*b*进行中序线索化，并返回线索化后头结点的指针root。
- Thread(p)算法：对以*p*为根结点的二叉树子树的中序线索化。

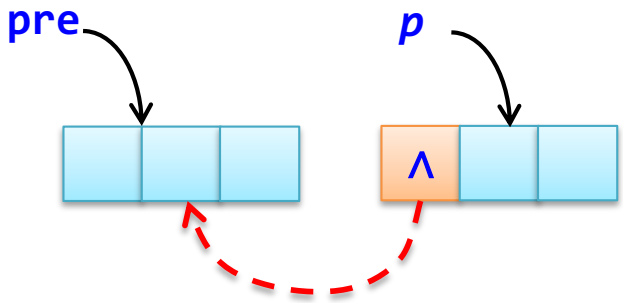
线索化二叉树结点类型

```
struct BthNode                          //线索二叉树结点类型
{   char data;                          //存放结点值
    BthNode* lchild,*rchild;            //左、右孩子或线索的指针
    int ltag,rtag;                      //左、右标记
    BthNode() {}                        //构造函数
    BthNode(char d)                     //重载构造函数
    {   data=d;
        ltag=rtag=0;
        lchild=rchild=NULL;
    }
};
```

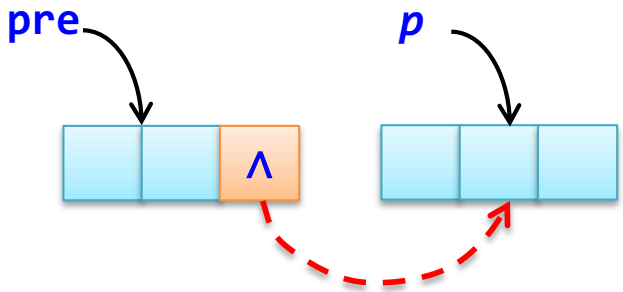
中序线索化二叉树类ThreadTree

```
class ThreadTree                                    //中序线索二叉树类
{  BthNode* r;                                      //二叉树的根结点指针
   BthNode* root;                                   //线索二叉树的头结点指针
   BthNode* pre;                                    //用于中序线索化,指向中序前驱结点
public:
   ThreadTree()                                     //构造函数,用于初始化
   {  r=NULL;                                       //初始二叉树为空树
      root=NULL;                                    //初始线索二叉树为空树
   }
   ~ThreadTree()                          //析构函数,用于释放线索二叉树的所有结点
   {  if (r!=NULL) DestroyBTree1(r); //释放原二叉树的所有结点
      if (root!=NULL) delete root;    //释放头结点
   }
   //二叉树的基本操作(结点类型改为BthNode)
   void CreateBTree(string str)  //创建以r为根结点的二叉链存储结构
   void DispBTree()              //输出二叉树的括号表示串
   //中序线索二叉树的基本操作
   void CreateThread()           //建立以root为头结点的中序线索二叉树
   void ThInOrder();             //中序线索二叉树的中序遍历
};
```

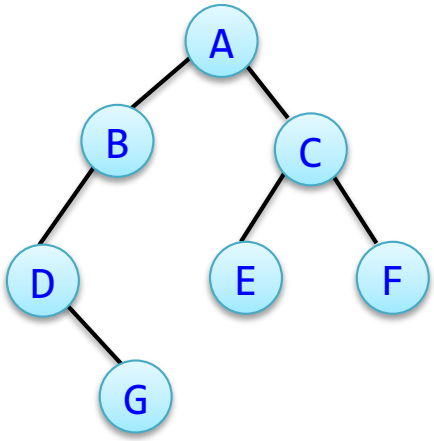
- 采用中序遍历进行中序线索化
- 在整个算法中*p*总是指向当前访问的结点，pre指向其前驱结点。



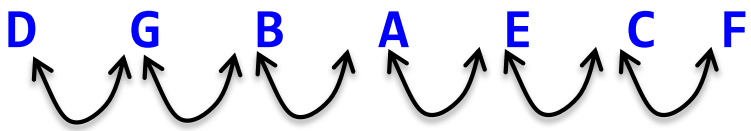
(a) 将结点*p*的左空指针改为线索



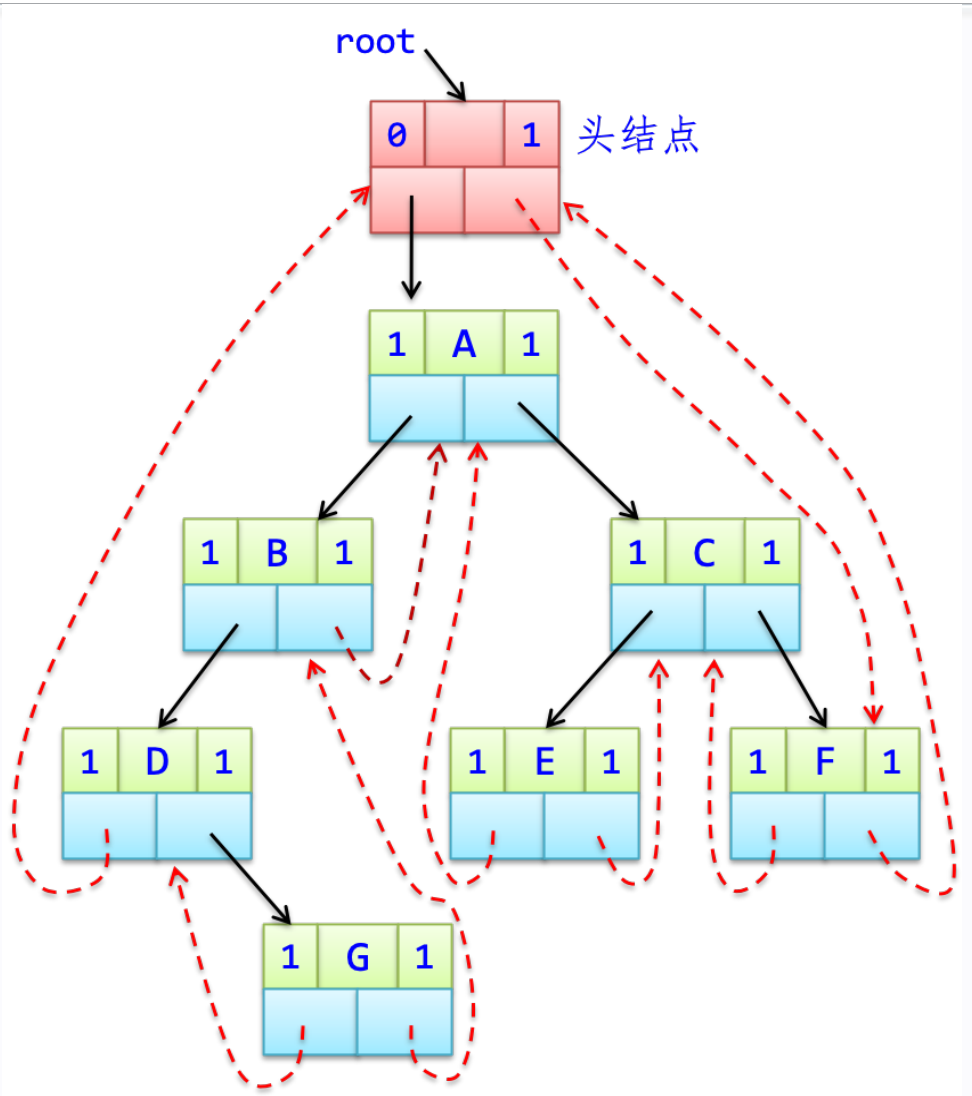
(b) 将结点pre的右空指针改为线索



中序序列：



```
void CreateThread()
{  root=new BthNode();
   root->ltag=0; root->rtag=1;
   root->rchild=r;
   if (r==NULL)
   {  root->lchild=root;
      root->rchild=NULL;
   }
   else
   {  root->lchild=r;
      pre=root;
      Thread(r);
      pre->rchild=root;
      pre->rtag=1;
      root->rchild=pre;
   }
}
```



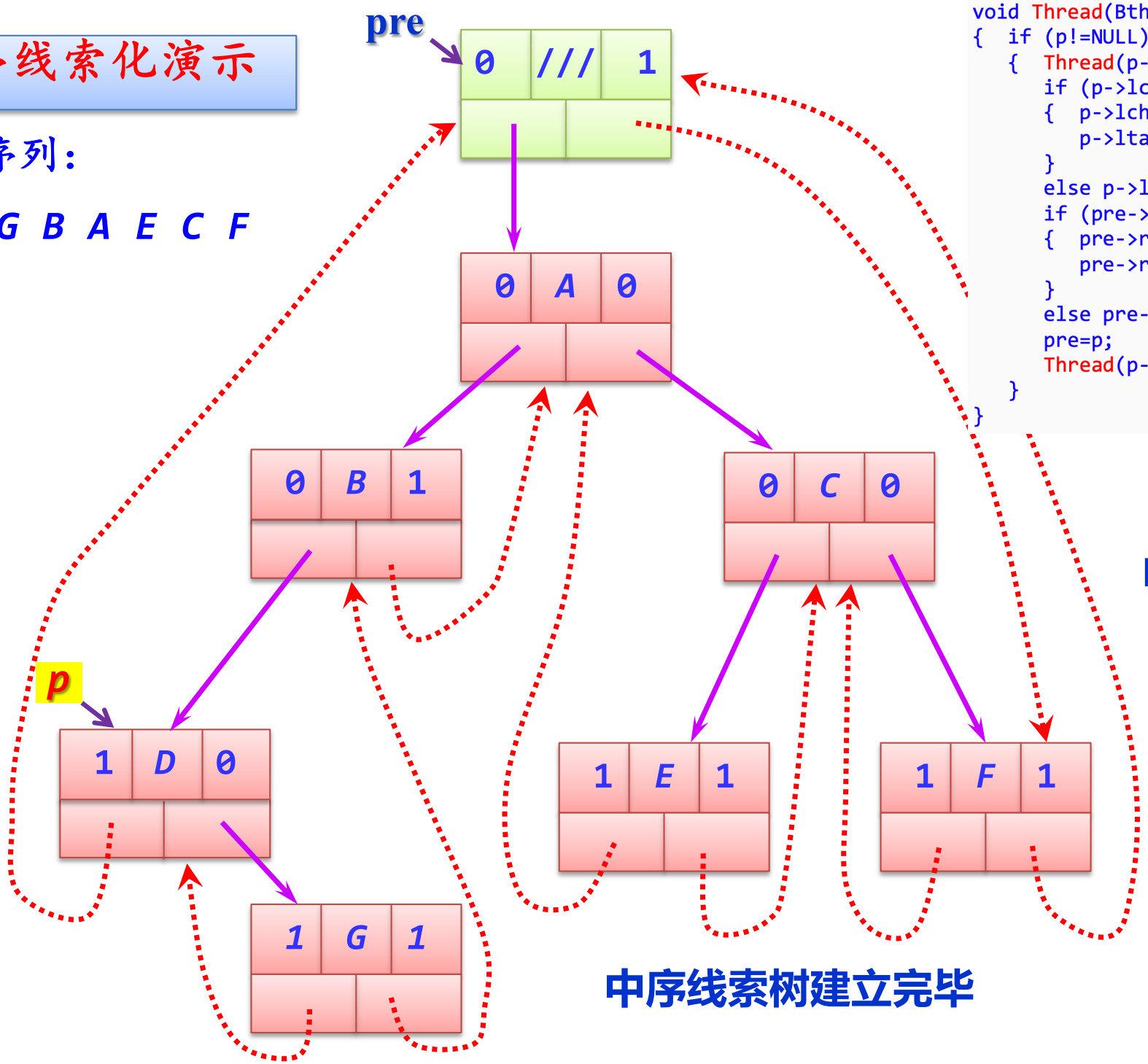
```
void Thread(BthNode*& p)                    //对结点p的二叉树中序线索化
{  if (p!=NULL)
   {  Thread(p->lchild);                    //左子树线索化
      if (p->lchild==NULL)                  //前驱线索
      {  p->lchild=pre;                     //给结点p添加前驱线索
         p->ltag=1;
      }
      else p->ltag=0;
      if (pre->rchild==NULL)
      {  pre->rchild=p;                     //给结点pre添加后继线索
         pre->rtag=1;
      }
      else pre->rtag=0;
      pre=p;
      Thread(p->rchild);                    //右子树线索化
   }
}
```

建立pre和p之间的线索：
相当于中序遍历中访问结点

中序线索化演示

中序序列：

D G B A E C F




```
void Thread(BthNode*& p)
{  if (p!=NULL)
   {  Thread(p->lchild);
      if (p->lchild==NULL)
      {  p->lchild=pre;
         p->ltag=1;
      }
      else p->ltag=0;
      if (pre->rchild==NULL)
      {  pre->rchild=p;
         pre->rtag=1;
      }
      else pre->rtag=0;
      pre=p;
      Thread(p->rchild);
   }
}
```

7.6.3 遍历线索化二叉树

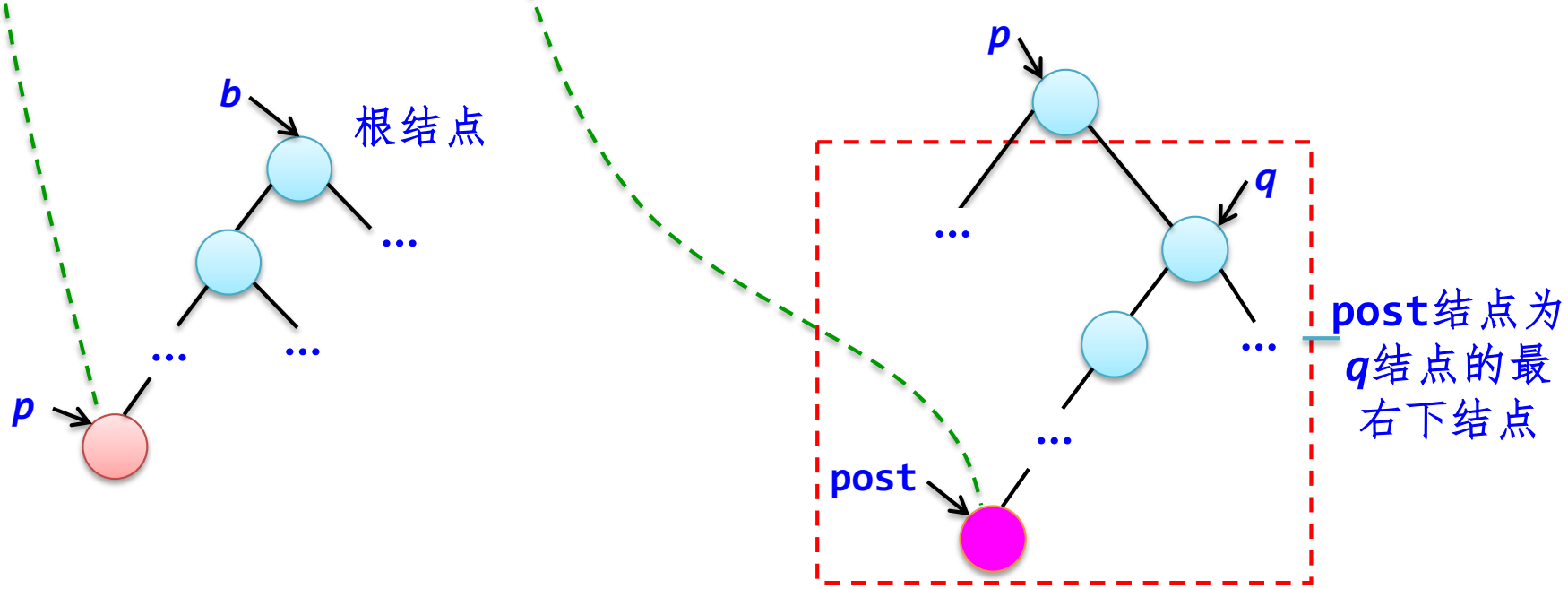
在该线索二叉树中实现中序遍历的两个步骤是：

（1）求中序序列的开始结点：实际上该结点就是根结点的最左下结点。

（2）对于一个结点*p*，求其后继结点的过程是：

① 如果*p*结点的rchild指针为线索，则rchild所指为其后继结点。

② 否则*p*结点的后继结点是其右孩子*q*的最左下结点post。



```
void ThInOrder()                                    //中序线索二叉树的中序遍历
{  BthNode* p=root->lchild;                         //p指向根结点
   while (p!=root)
   {  while (p!=root && p->ltag==0)
         p=p->lchild;                               //找开始结点p
      cout << p->data;                              //访问结点p
      while (p->rtag==1 && p->rchild!=root)  //如果是线索，一直找下去
      {  p=p->rchild;
         cout << p->data;                           //访问结点p
      }
      p=p->rchild;                                  //如果不再是线索，转向其右子树
   }
}
```

该算法是一个非递归算法，算法的时间复杂度为O(*n*)，空间复杂度为O(1)

（1）求中序序列的开始结点：实际上该结点就是根结点的最左下结点。

（2）对于一个结点*p*，求其后继结点的过程是：

① 如果*p*结点的rchild指针为线索，则rchild所指为其后继结点。

② 否则*p*结点的后继结点是其右孩子*q*的最左下结点post。

线索二叉树的中序遍历示例演示

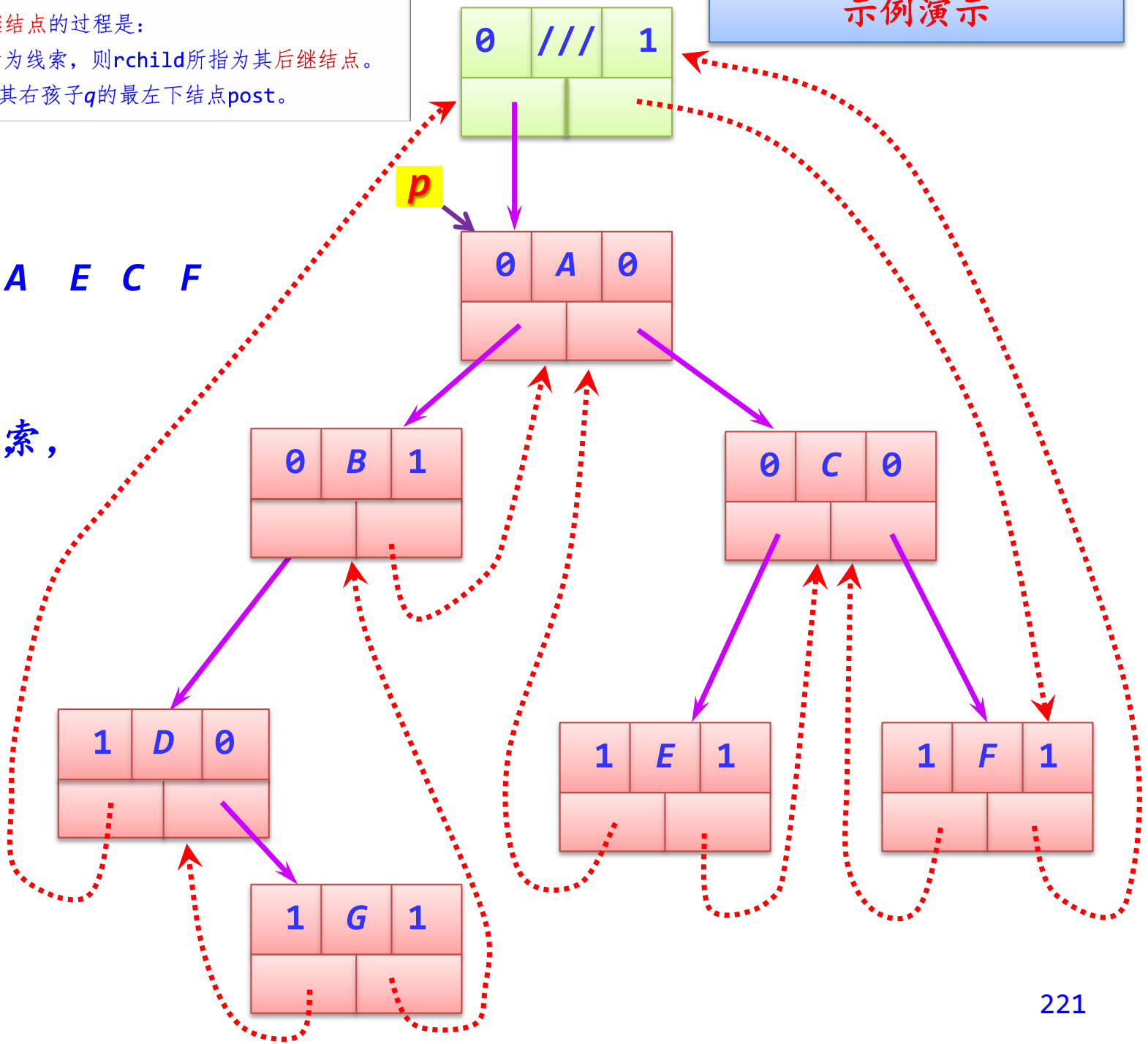
中序序列：

D G B A E C F

操作：

找开始结点，

沿着线索访问



优化编码

对于英语句子来说，ASCII码表示可能相当浪费，因为在英语句子中字母出现的频率各不相同。如果字母 “e” 出现得如此频繁，也许我们应该有一种更高效的方式来表示它（例如，用更少的比特位来表示 “e”）！

everyday english sentence

```
01100101 01110110 01100101 01110010 01111001 01100100 01100001
01111001 00100000 01100101 01101110 01100111 01101100 01101001
01110011 01101000 00100000 01110011 01100101 01101110 01110100
         01100101 01101110 01100011 01100101
```

输入：字符的某种分布情况（各个字符出现的频率）
输出：一种尽可能高效的字符编码方式

优化编码

ASCII can be pretty wasteful for English sentences, where letters have varying frequencies. If **e** shows up so often, maybe we should have a more efficient way of representing it (e.g. use less bits to represent **e**)!

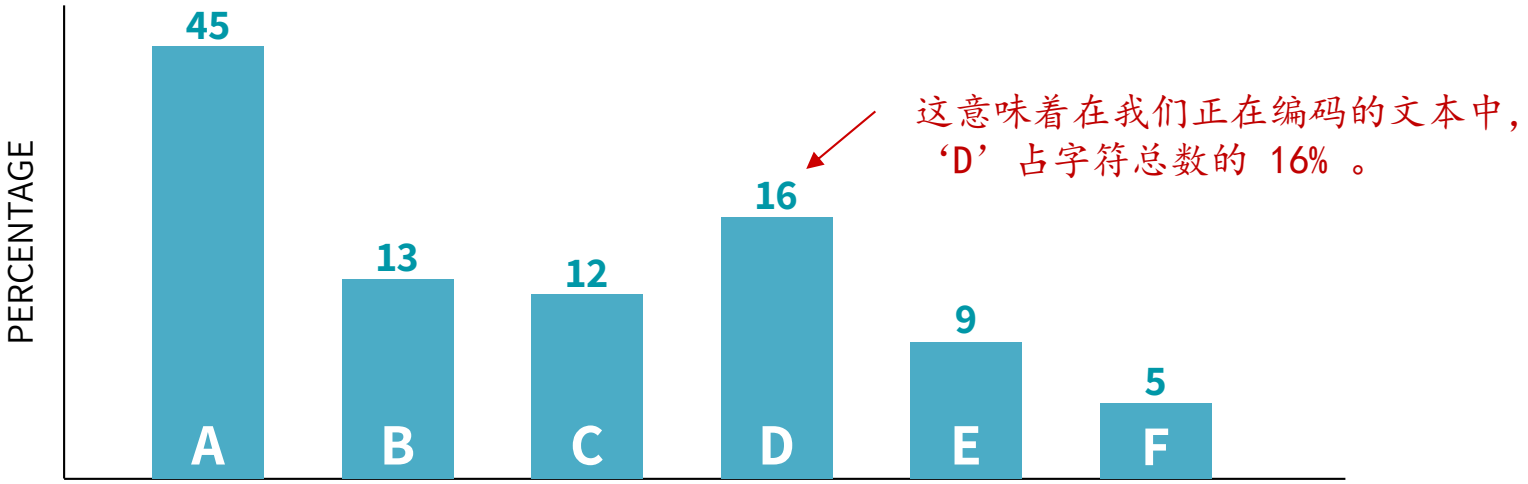
everyday english sentence

01100101 01110110 **01100101** 01110010 01111001 01100100 01100001
01111001 00100000 **01100101** 01101110 01100111 01101100 01101001
01110011 01101000 00100000 01110011 **01100101** 01101110 01110100
01100101 01101110 01100011 **01100101**

Input: Some distribution on characters (frequencies of characters)
Output: A way to encode the characters as efficiently* as possible

优化编码

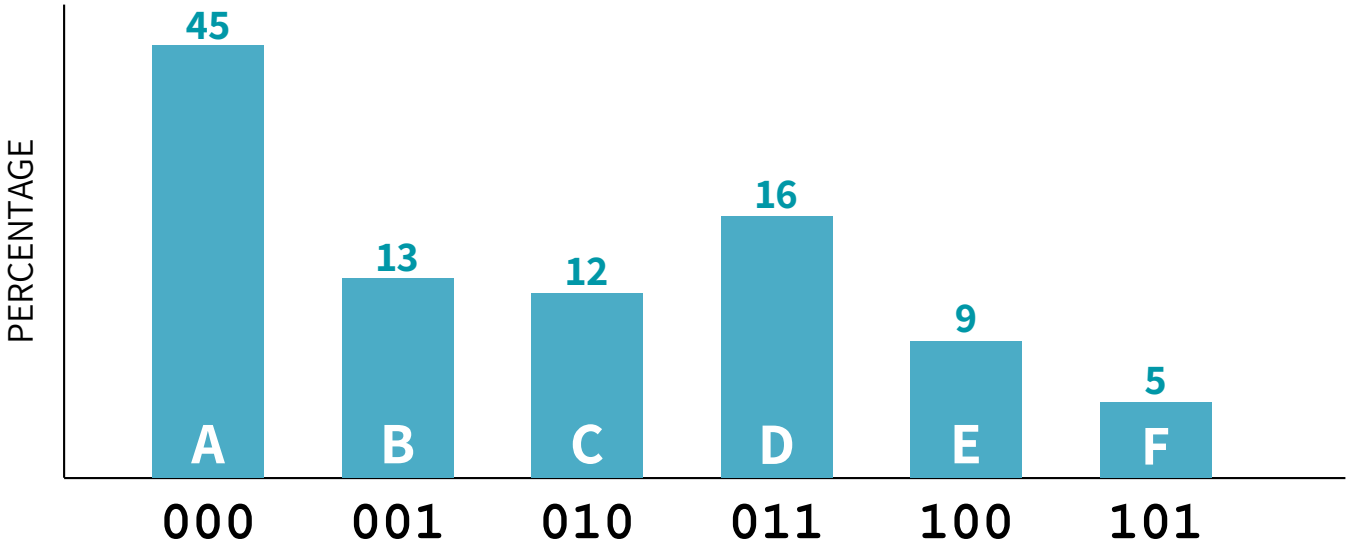
输入： 字符的某种分布情况（各个字符出现的频率）
输出： 一种尽可能高效的字符编码方式



目标： 使用于对一个符号进行编码的平均比特位数最小化（符号根据其出现频率进行加权计算）。

优化编码：尝试0

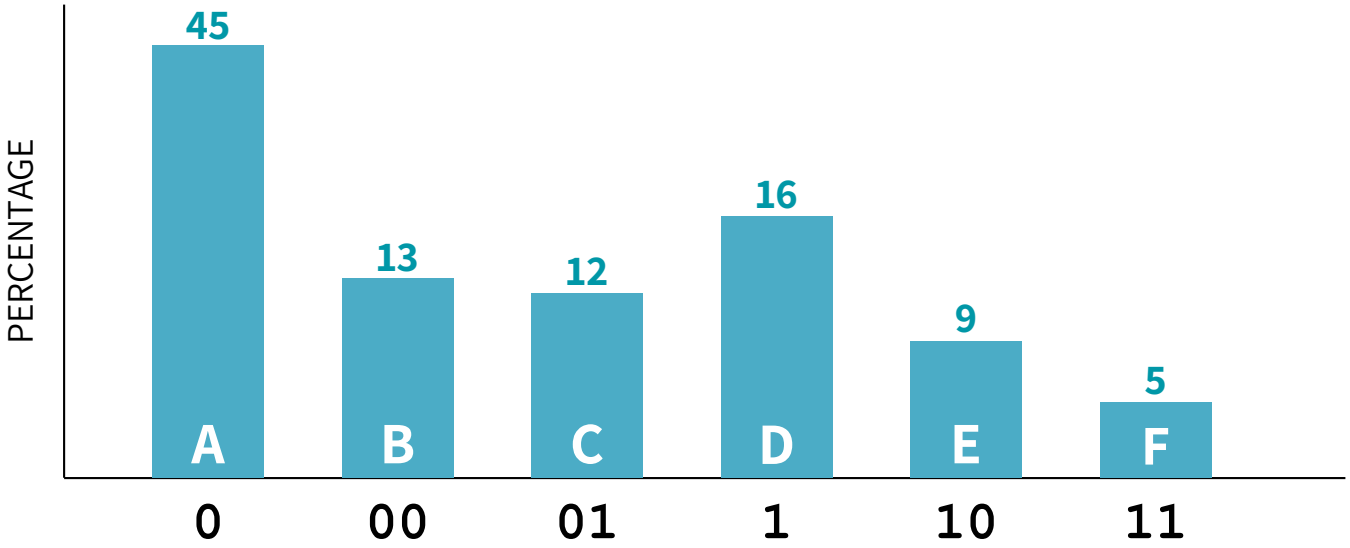
尝试0：使用固定长度编码（第i个字符被编码为i的二进制表示）



我们确实应该尝试用更少的比特表示更常见的符号……

优化编码：尝试1

尝试 1：采用变长编码（常见字符使用更短的编码）

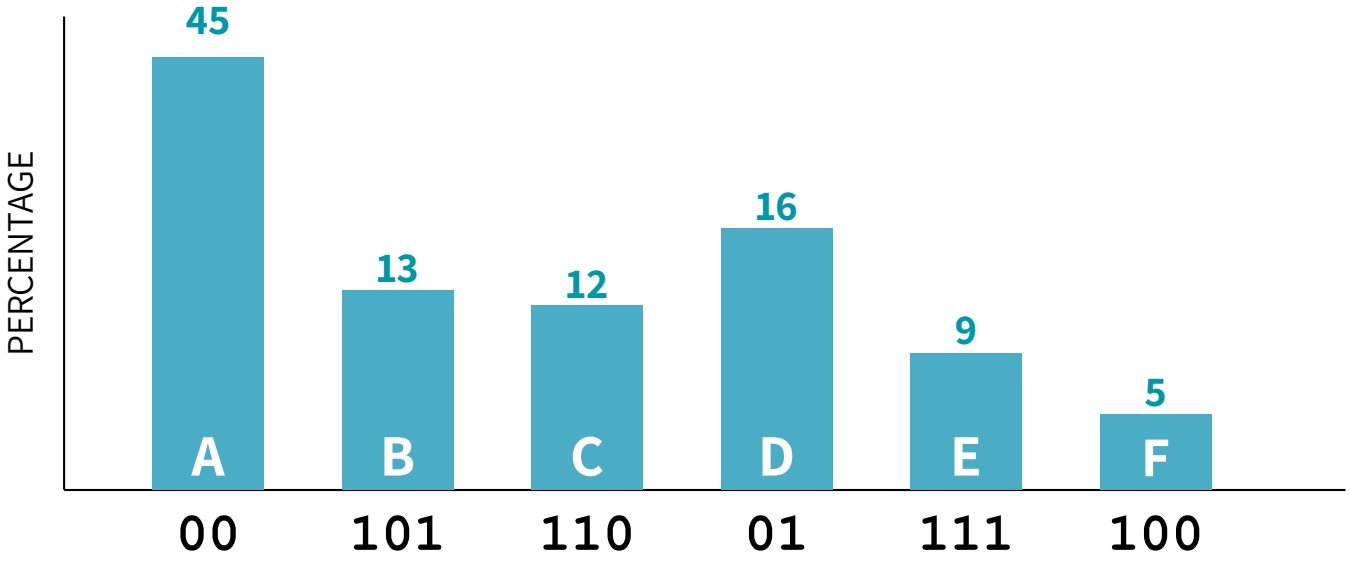


What is 001?

可能是 “AC”，也可能是 “BD”…… 我们引入了歧义！

优化编码：尝试2

尝试 2：使用变长前缀无关码（即任何字符的编码都不会成为其他字符编码的前缀，有时称为前缀码）



What is 0011001?

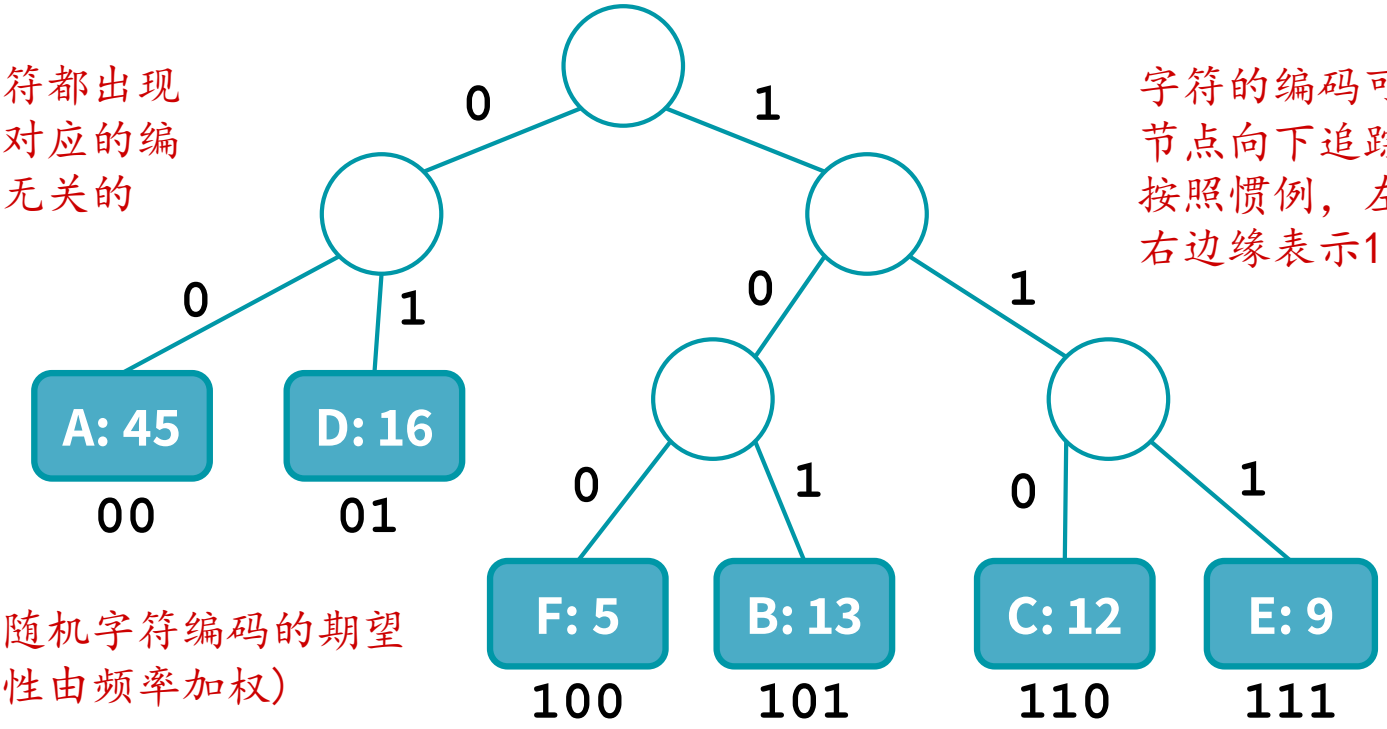
A C D

前缀无关编码(PREFIX-FREE CODE)是一棵树

我们可以将前缀无关码视为一棵树

只要所有字符都出现在叶节点，对应的编码就是前缀无关的

字符的编码可以通过从根节点向下追踪路径获得。按照惯例，左边缘表示0，右边缘表示1



树的成本是随机字符编码的期望长度（随机性由频率加权）

前缀无关编码(PREFIX-FREE CODE)是一棵树

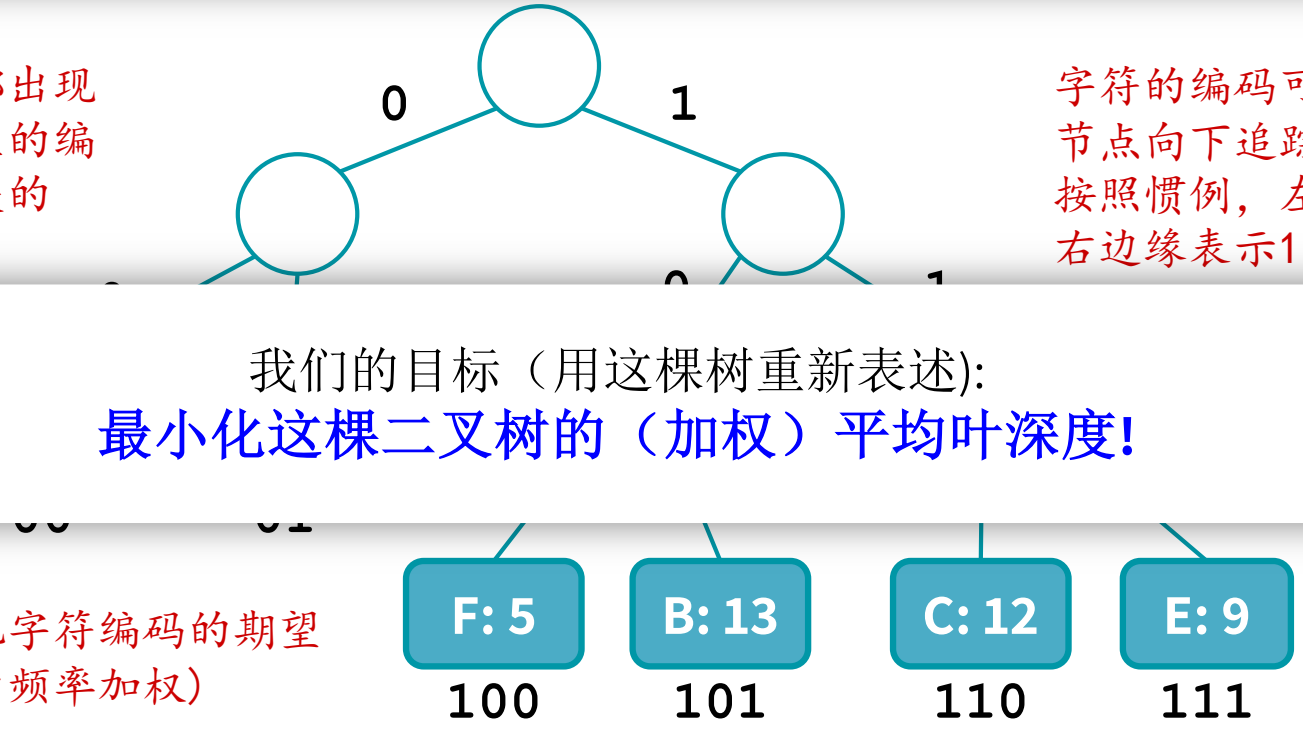
这棵树的成本（平均叶深度）：

$$\sum_{\text{leaves } x} P(x) \cdot \text{depth}(x)$$

$(2 \cdot 0.45) + (2 \cdot 0.16) + (3 \cdot 0.05) + (3 \cdot 0.13) + (3 \cdot 0.12) + (3 \cdot 0.09) = \mathbf{2.39}$

只要所有字符都出现在叶节点，对应的编码就是前缀无关的

字符的编码可以通过从根节点向下追踪路径获得。按照惯例，左边缘表示0，右边缘表示1



我们的目标（用这棵树重新表述):

最小化这棵二叉树的（加权）平均叶深度!

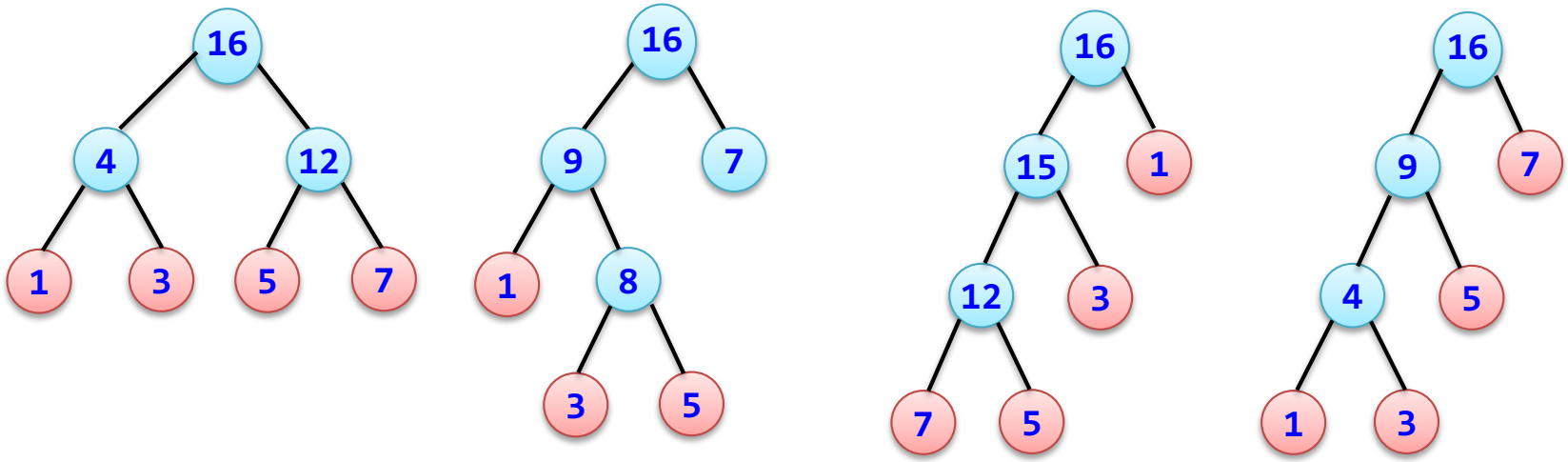
树的成本是随机字符编码的期望长度（随机性由频率加权）

7.7 哈夫曼树

7.7.1 哈夫曼树的定义

- 应用中常给树中的结点赋上一个有着某种意义的数值—权。
- 从树根结点到某个结点之间的路径长度与该结点权的乘积称为结点的带权路径长度。
- 一棵二叉树树中所有叶子结点的带权路径长度之和称为该树的带权路径长度，通常记 $\mathrm{WPL}=\sum_{i=1}^{n_0} w_i \times l_i$
- 在n_0个带权叶子结点构成的所有二叉树中，带权路径长度WPL最小的二叉树称为哈夫曼树（或最优二叉树）。

给定4个叶子结点，设其权值分别为1、3、5、7，可以构造这样的4棵二叉树。



（a）　（b）　（c）　（d）

（a）WPL=1×2+3×2+5×2+7×2=32

（b）WPL=1×2+3×3+5×3+7×1=33

（c）WPL=7×3+5×3+3×2+1×1=43

（d）WPL=1×3+3×3+5×2+7×1=29　√

7.7.2 哈夫曼树的构造算法

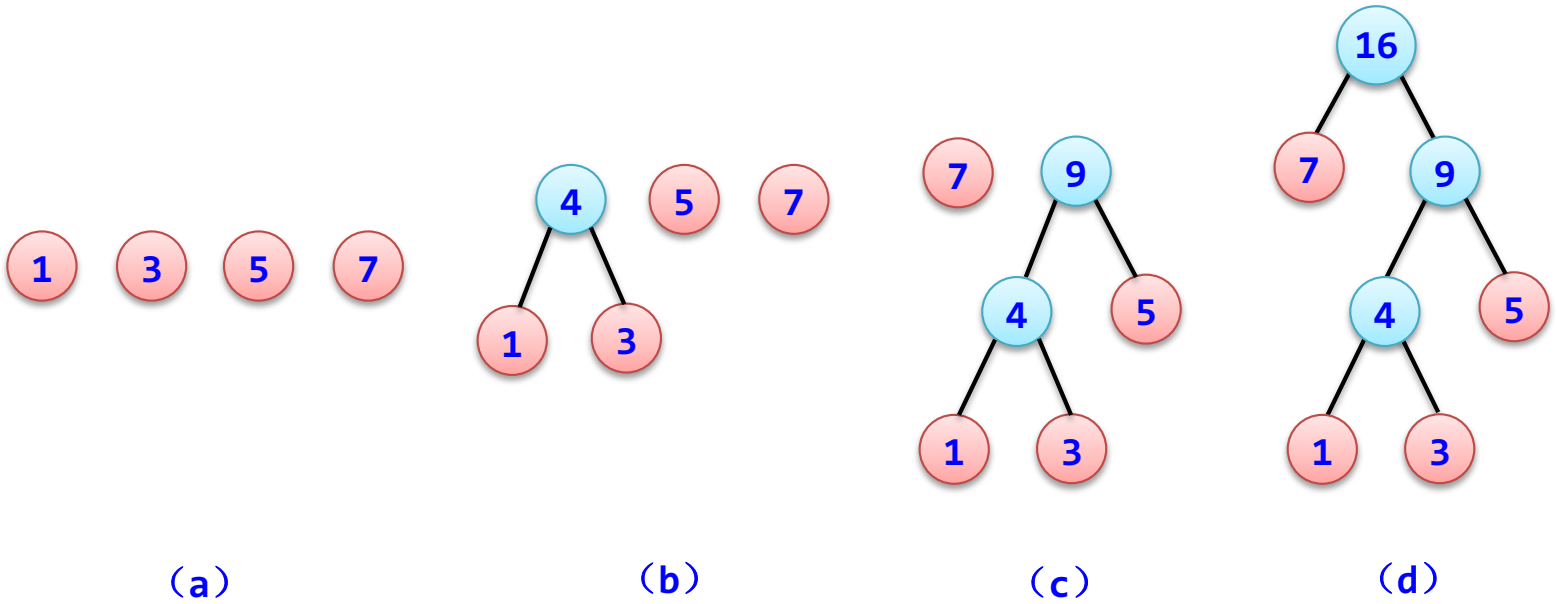
（1）根据给定的n_0个权值$W=(w_1, w_2, \cdots, w_{n0})$，对应结点构成$n_0$棵二叉树的森林$T=(T_1, T_2, \cdots, T_{n0})$，其中每棵二叉树$T_i$（$1 \leqslant i \leqslant n_0$）中都只有一个带权值为$w_i$的根结点，其左、右子树均为空。

（2）在森林T中选取两棵根结点权值最小的子树作为左、右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为其左、右子树上根的权值之和。称为合并，每合并一次T中减少一棵二叉树。

（3）重复（2）直到T只含一棵树为止。这棵树便是哈夫曼树。

每次都是两棵子树合并 ⇨ 哈夫曼树中没有单分支结点

W=(1，3，5，7)来构造一棵哈夫曼树



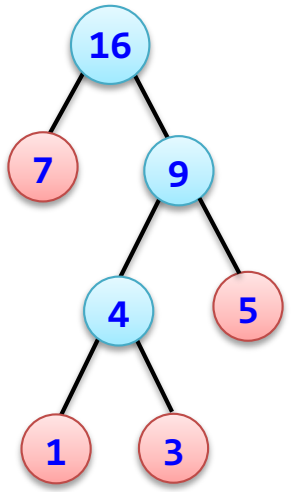
（a）　（b）　（c）　（d）

定理7.3 对于具有n_0个叶子结点的哈夫曼树，共有$2n_0-1$个结点。

证明：从哈夫曼树的构造过程看出，每次合并都是将两棵二叉树合并为一个，所以哈夫曼树不存在度为1的结点，即$n_1=0$。

由二叉树的性质1可知$n_0=n_2+1$，即$n_2=n_0-1$

则结点总数$n=n_0+n_1+n_2= n_0+n_2=n_0+2n_0-1=2n_0-1$。



（d）

构造哈夫曼树中采用静态数组ht存储哈夫曼树，即每个数组元素存放一个结点。设计哈夫曼树中结点类如下：

```
struct HTNode                       //哈夫曼树结点类
{  char data;                       //结点值
   double weight;                   //权值
   int parent;                      //双亲结点
   int lchild;                      //左孩子结点
   int rchild;                      //右孩子结点
   bool flag;                       //标识是双亲的左(true)或者右(false)孩子

   HTNode()                         //构造函数
   {  parent=-1;
      lchild=rchild=-1;
   }
   HTNode(char d,double w)          //重载构造函数
   {  data=d;
      weight=w;
      parent=lchild=rchild=-1;
      flag=true;
   }
};
```

构造哈夫曼树中采用静态数组ht存储哈夫曼树，即每个数组元素存放一个结点。设计哈夫曼树中结点类如下：

```
struct HTNode              //哈夫曼树结点类
{   char data;             //结点值
    double weight;         //权值
    int parent;            //双亲结点
    int lchild;            //左孩子结点
    int rchild;            //右孩子结点
    bool flag;             //标识是双亲的左(true)或者右(false)孩子
};

struct HeapNode            //优先队列元素类型
{   double w;              //权值
    int i;                 //对应哈夫曼树中结点编号
};
```

```
struct HeapNode                                  //优先队列元素类型
{  double w;                                     //权值
   int i;                                        //对应哈夫曼树中结点编号

   HeapNode(double w1,int i1):w(w1),i(i1) {} //构造函数
   bool operator<(const HeapNode& s) const
   {                                             //将当前对象跟对象s进行比较
      return w>s.w;                              //按w越小越优先出队
   }
};
```

- STL中的优先队列是priority_queue容器，它和stack/queue一样，它也是一种适配器容器，其底层容器必须是用数组实现的，可以是vector（默认）或者deque，不能是list。
- priority_queue对象的一般定义格式如下：

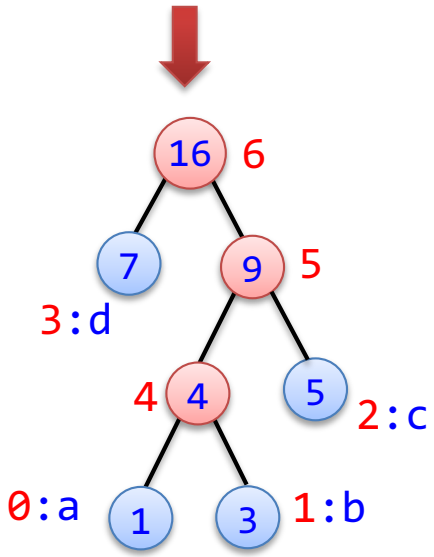
```
priority_queue<type, container, functional>
```

```
struct HeapNode                                  //优先队列元素类型
{  double w;                                     //权值
   int i;                                        //对应哈夫曼树中结点编号

   HeapNode(double w1,int i1):w(w1),i(i1) {} //构造函数
   bool operator<(const HeapNode& s) const
   {                                             //将当前对象跟对象s进行比较
      return w>s.w;                              //按w越小越优先出队
   }
};
```

```
void CreateHT()                                  /
{  priority_queue<HeapNode> qu;                  /                堆)
   for (int i=0;i<n0;i++)     //i从0到r                  结点并进队
   {  ht[i]=HTNode(D[i],W[i]);                   /
      qu.push(HeapNode(W[i],i));                 /
   }
   for (int i=n0;i<2*n0-1;i++)                   /                n0-1次合并操作
   {  HeapNode p1=qu.top(); qu.pop();//出队两个权值最小的元素p1和p2
      HeapNode p2=qu.top(); qu.pop();
      ht[i]=HTNode();                            //新建ht[i]结点
      ht[i].weight=ht[p1.i].weight+ht[p2.i].weight;//求权值和
      ht[p1.i].parent=i;                         //设置ht[p1.i]的双亲为ht[i]
      ht[i].lchild=p1.i;                         //将ht[p1.i]作为双亲ht[i]的左孩子
      ht[p1.i].flag=true;
      ht[p2.i].parent=i;                         //设置ht[p2.i]的双亲为ht[i]
      ht[i].rchild=p2.i;                         //将ht[p2.i]作为双亲ht[i]的右孩子
      ht[p2.i].flag=false;
      qu.push(HeapNode(ht[i].weight,i));     //将新结点ht[i]进队
   }
}
```

```
struct HTNode
{  char data;
   double weight;
   int parent;
   int lchild;
   int rchild;
   bool flag;
};
```

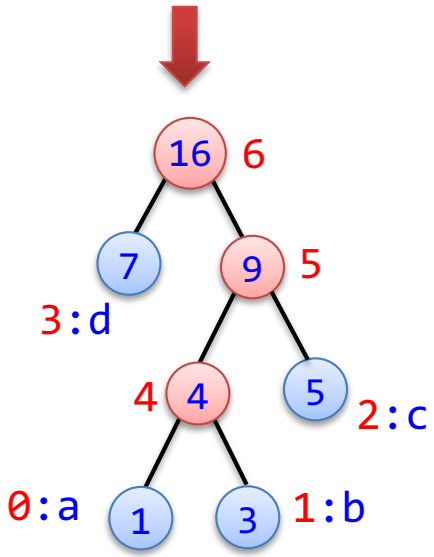
n_0=4, D=['a','b','c','d'], W=[1,3,5,7]




```
void CreateHT()                          //构造哈夫曼树
{  priority_queue<HeapNode> qu;          //建立优先队列(w小根堆)
   for (int i=0;i<n0;i++)     //i从0到n0-1循环建立n0个叶子结点并进队
   {  ht[i]=HTNode(D[i],W[i]);           //建立一个叶子结点
      qu.push(HeapNode(W[i],i));         //将(W[i],i)进队
   }
   for (int i=n0;i<2*n0-1;i++)           //i从n0到2n0-2循环做n0-1次合并操作
   {  HeapNode p1=qu.top(); qu.pop();//出队两个权值最小的元素p1和p2
      HeapNode p2=qu.top(); qu.pop();
      ht[i]=HTNode();                    //新建ht[i]结点
      ht[i].weight=ht[p1.i].weight+ht[p2.i].weight;//求权值和
      ht[p1.i].parent=i;                 //设置ht[p1.i]的双亲为ht[i]
      ht[i].lchild=p1.i;                 //将ht[p1.i]作为双亲ht[i]的左孩子
      ht[p1.i].flag=true;
      ht[p2.i].parent=i;                 //设置ht[p2.i]的双亲为ht[i]
      ht[i].rchild=p2.i;                 //将ht[p2.i]作为双亲ht[i]的右孩子
      ht[p2.i].flag=false;
      qu.push(HeapNode(ht[i].weight,i));     //将新结点ht[i]进队
   }
}
```

i（结点索引）	0	1	2	3
D[i]	a	b	c	d
W[i]	1	3	5	7
parent	-1	-1	-1	-1
lchild	-1	-1	-1	-1
rchild	-1	-1	-1	-1
flag	true	true	true	true

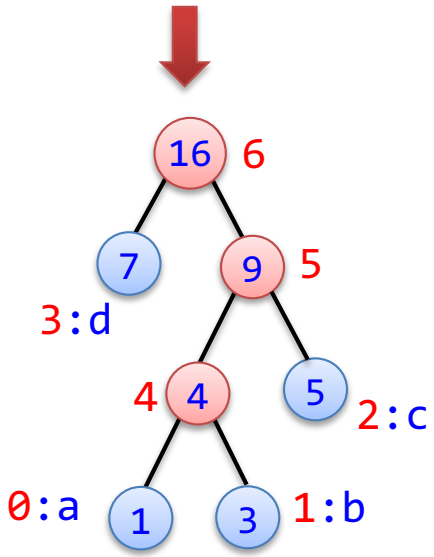
n_0=4, D=['a','b','c','d'], W=[1,3,5,7]




```
void CreateHT()                          //构造哈夫曼树
{  priority_queue<HeapNode> qu;          //建立优先队列(w小根堆)
   for (int i=0;i<n0;i++)     //i从0到n0-1循环建立n0个叶子结点并进队
   {  ht[i]=HTNode(D[i],W[i]);           //建立一个叶子结点
      qu.push(HeapNode(W[i],i));         //将(W[i],i)进队
   }
   for (int i=n0;i<2*n0-1;i++)           //i从n0到2n0-2循环做n0-1次合并操作
   {  HeapNode p1=qu.top(); qu.pop();//出队两个权值最小的元素p1和p2
      HeapNode p2=qu.top(); qu.pop();
      ht[i]=HTNode();                    //新建ht[i]结点
      ht[i].weight=ht[p1.i].weight+ht[p2.i].weight;//求权值和
      ht[p1.i].parent=i;                 //设置ht[p1.i]的双亲为ht[i]
      ht[i].lchild=p1.i;                 //将ht[p1.i]作为双亲ht[i]的左孩子
      ht[p1.i].flag=true;
      ht[p2.i].parent=i;                 //设置ht[p2.i]的双亲为ht[i]
      ht[i].rchild=p2.i;                 //将ht[p2.i]作为双亲ht[i]的右孩子
      ht[p2.i].flag=false;
      qu.push(HeapNode(ht[i].weight,i));     //将新结点ht[i]进队
   }
}
```

i（结点索引）	0	1	2	3	4	5	6
D[i]	a	b	c	d			
W[i]	1	3	5	7			
parent	-1	-1	-1	-1			
lchild	-1	-1	-1	-1			
rchild	-1	-1	-1	-1			
flag	true	true	true	true			

n_0=4, D=['a','b','c','d'], W=[1,3,5,7]

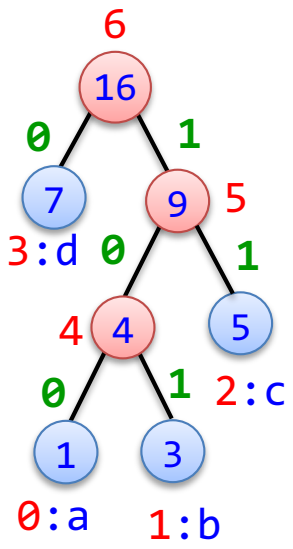



```
void CreateHT()                          //构造哈夫曼树
{  priority_queue<HeapNode> qu;          //建立优先队列(w小根堆)
   for (int i=0;i<n0;i++)     //i从0到n0-1循环建立n0个叶子结点并进队
   {  ht[i]=HTNode(D[i],W[i]);           //建立一个叶子结点
      qu.push(HeapNode(W[i],i));         //将(W[i],i)进队
   }
   for (int i=n0;i<2*n0-1;i++)           //i从n0到2n0-2循环做n0-1次合并操作
   {  HeapNode p1=qu.top(); qu.pop();//出队两个权值最小的元素p1和p2
      HeapNode p2=qu.top(); qu.pop();
      ht[i]=HTNode();                    //新建ht[i]结点
      ht[i].weight=ht[p1.i].weight+ht[p2.i].weight;//求权值和
      ht[p1.i].parent=i;                 //设置ht[p1.i]的双亲为ht[i]
      ht[i].lchild=p1.i;                 //将ht[p1.i]作为双亲ht[i]的左孩子
      ht[p1.i].flag=true;
      ht[p2.i].parent=i;                 //设置ht[p2.i]的双亲为ht[i]
      ht[i].rchild=p2.i;                 //将ht[p2.i]作为双亲ht[i]的右孩子
      ht[p2.i].flag=false;
      qu.push(HeapNode(ht[i].weight,i));     //将新结点ht[i]进队
   }
}
```

i（结点索引）	0	1	2	3	4	5	6
D[i]	a	b	c	d			
W[i]	1	3	5	7	4	9	16
parent	4	4	5	6	5	6	-1
lchild	-1	-1	-1	-1	0	4	3
rchild	-1	-1	-1	-1	1	2	5
flag	true	false	false	true	true	false	false

7.7.3 哈夫曼编码

- 构造一棵哈夫曼树。
- 规定哈夫曼树中的左分支为0，右分支为1。
- 从根结点到每个叶子结点所经过的分支对应的0和1组成的序列便为该结点对应字符的编码。这样的编码称为哈夫曼编码。

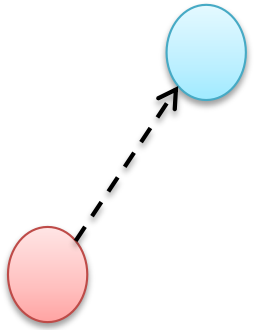


```
a: 100
b: 101
c: 11
d: 0
```

哈夫曼编码的实质就是使用频率越高的采用越短的编码。

只有ht[0..n_0-1]叶子结点才对应哈夫曼编码，用hcd[i]（0≤i≤n_0-1）表示ht[i]叶子结点的哈夫曼编码。

```
void CreateHCode()                  //根据哈夫曼树求哈夫曼编码
{  for (int i=0;i<n0;i++)           //遍历下标从0到n0-1的叶子结点
   {  string code="";
      int j=i;                      //从ht[i]开始找双亲结点
      while (ht[j].parent!=-1)
      {  if (ht[j].flag)                  //ht[j]结点是双亲的左孩子
            code+="0";
         else                             //ht[j]结点是双亲的右孩子
            code+="1";
         j=ht[j].parent;
      }
      reverse(code.begin(),code.end()); //将code逆置并添加到hcd中
      hcd[i]=code;
   }
}
```

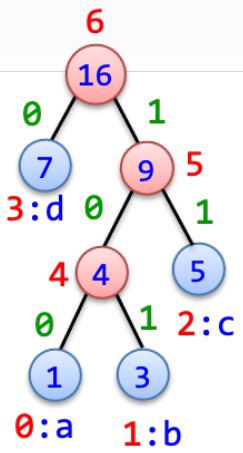


向根结点的方向求编码，
再逆置

叶子结点ht[0..n_0-1]

只有ht[0..n_0-1]叶子结点才对应哈夫曼编码，用hcd[i]（0≤i≤n_0-1）表示ht[i]叶子结点的哈夫曼编码。

```
void CreateHCode()                  //根据哈夫曼树求哈夫曼编码
{  for (int i=0;i<n0;i++)           //遍历下标从0到n0-1的叶子结点
   {  string code="";
      int j=i;                      //从ht[i]开始找双亲结点
      while (ht[j].parent!=-1)
      {  if (ht[j].flag)                  //ht[j]结点是双亲的左孩子
            code+="0";
         else                             //ht[j]结点是双亲的右孩子
            code+="1";
         j=ht[j].parent;
      }
      reverse(code.begin(),code.end()); //将code逆置并添加到hcd中
      hcd[i]=code;
   }
}
```



a: 100

b: 101

c: 11

d: 0

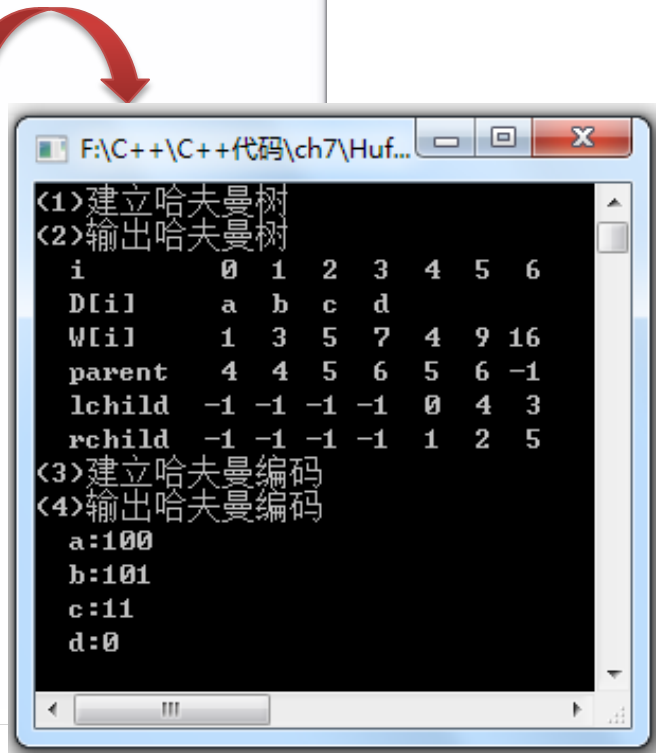
i（结点索引）	0	1	2	3	4	5	6
D[i]	a	b	c	d			
W[i]	1	3	5	7	4	9	16
parent	4	4	5	6	5	6	-1
lchild	-1	-1	-1	-1	0	4	3
rchild	-1	-1	-1	-1	1	2	5
flag	true	false	false	true	true	false	false

```
void DispHT()                                   //输出哈夫曼树
{  printf("  i      ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3d",i);
   printf("\n");
   printf("  D[i]   ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3c",ht[i].data);
   printf("\n");
   printf("  W[i]   ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3g",ht[i].weight);
   printf("\n");
   printf("  parent ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3d",ht[i].parent);
   printf("\n");
   printf("  lchild ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3d",ht[i].lchild);
   printf("\n");
   printf("  rchild ");
   for (int i=0;i<2*n0-1;i++)
      printf("%3d",ht[i].rchild);
   printf("\n");
}
```

```
void DispHCode()                    //输出哈夫曼编码
{  for (int i=0;i<n0;i++)
       cout << "  " << ht[i].data << ":" << hcd[i] << endl;
}
```

程序验证

```
int n0=4;                          //编码的字符个数
char D[]={'a','b','c','d'};        //字符列表
double W[]={1,3,5,7};              //权值列表
int main()
{  printf("(1)建立哈夫曼树\n");
   CreateHT();
   printf("(2)输出哈夫曼树\n");
   DispHT();
   printf("(3)建立哈夫曼编码\n");
   CreateHCode();
   printf("(4)输出哈夫曼编码\n");
   DispHCode();
   return 0;
}
```



在一组字符的哈夫曼编码中，任一字符的哈夫曼编码不可能是另一字符哈夫曼编码的前缀。

2014年全国硕士研究生入学统一考试题

6．5个字符有如下4种编码方案，不是前缀编码的是（ ）。

A．01,0000,0001,001,1

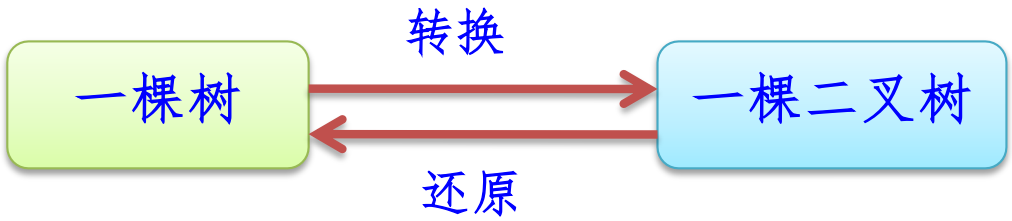
B．011,000,001,010,1

C．000,001,010,011,100

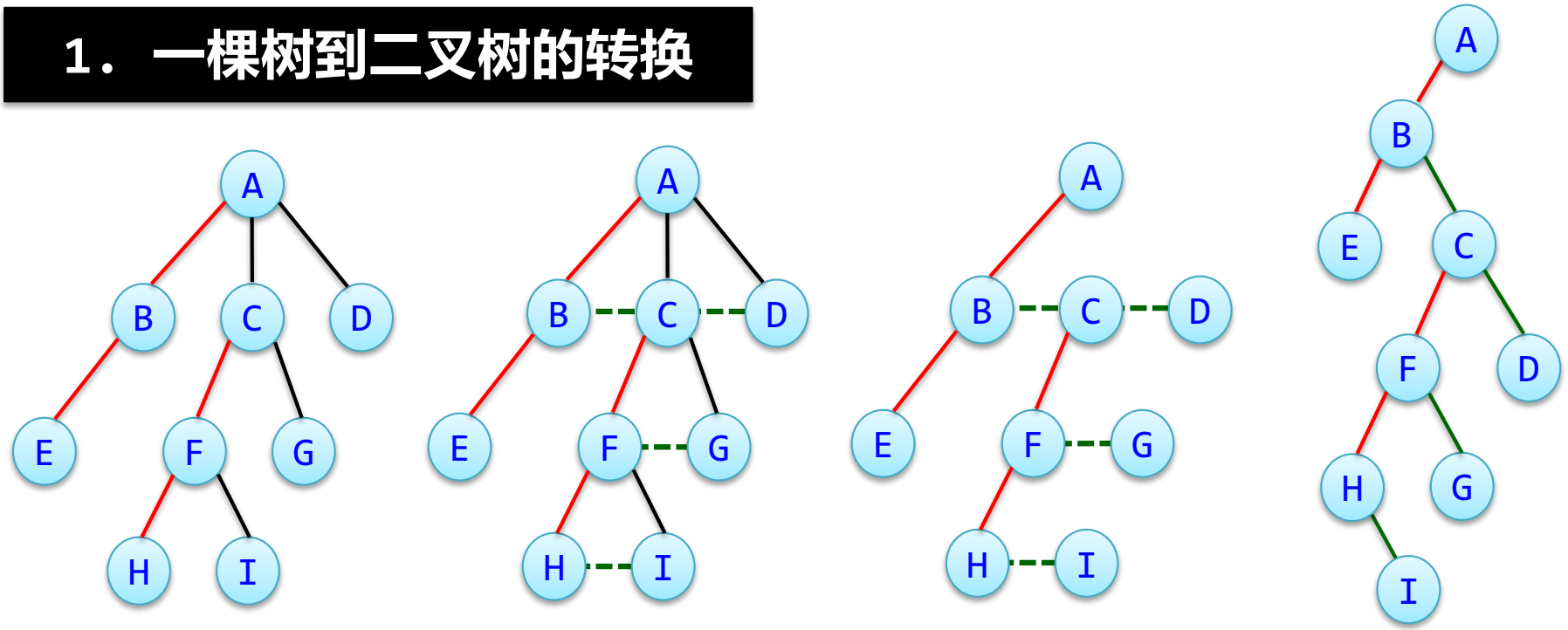
D．0,100,110,1110,1100

7.8 树/森林与二叉树的转换及还原

7.8.1 一棵树与二叉树的转换及还原



1．一棵树到二叉树的转换



（a）一棵树　（b）加线　（c）抹线　（d）调整

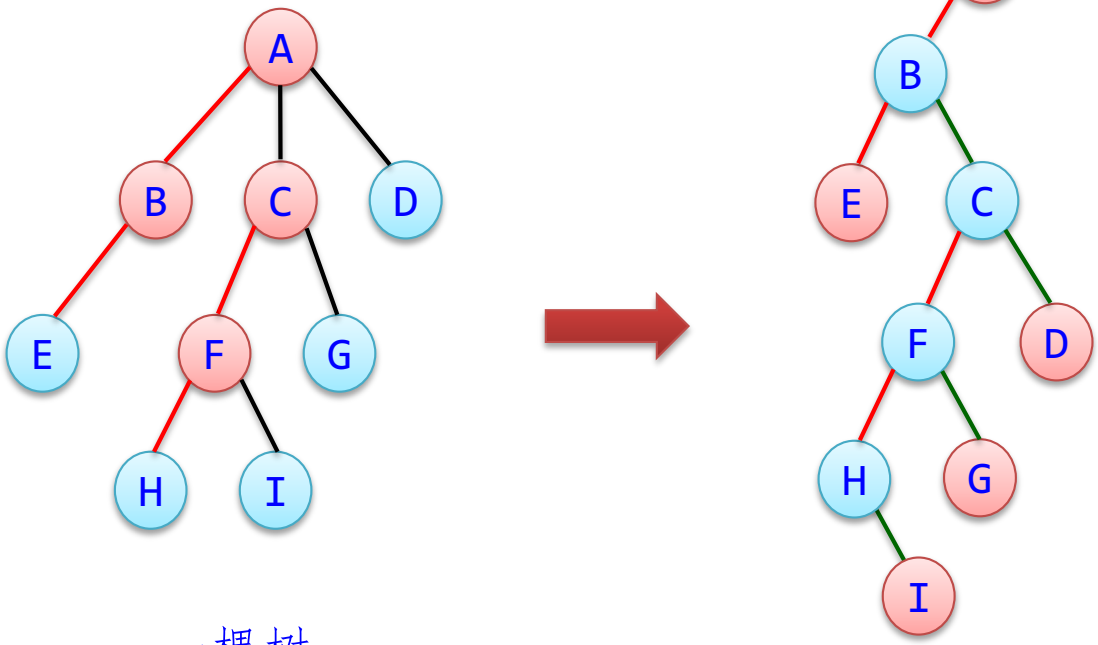
转换的特点

- 根结点只有左子树而没有右子树。
- 左分支不变（左分支为最左孩子），兄弟变成右分支（右分支实为双亲的兄弟）。

1．一棵树到二叉树的转换

一棵树可以按照以下规则转换为二叉树：

- 加线：在各兄弟结点之间加一连线，将其隐含的“兄—弟”关系以“双亲—右孩子”关系显示表示出来。
- 抹线：对任意结点，除了其最左子树之外，抹掉该结点与其他子树之间的“双亲—孩子”关系。
- 调整：以树的根结点作为二叉树的根结点，将树根与其最左子树之间的“双亲—孩子”关系改为“双亲—左孩子”关系，且将各结点按层次排列，形成二叉树。

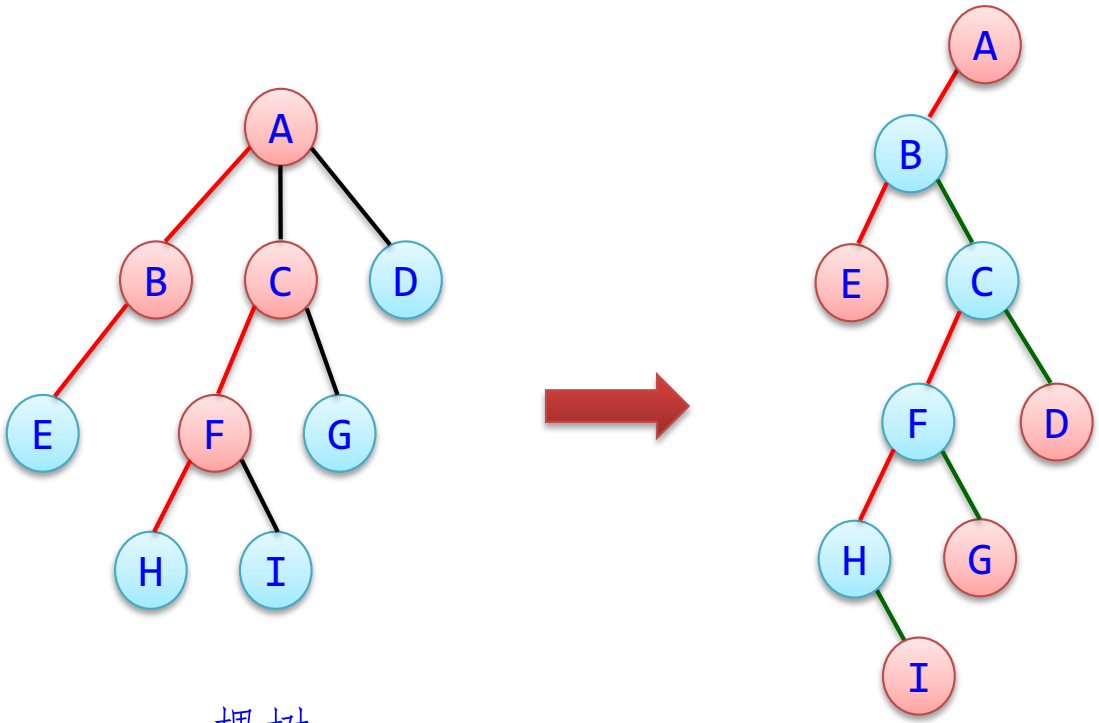


一棵树　　二叉树

扩展

树中分支结点个数为***n***，则二叉树中无右孩子的结点个数为***n*+1**。这里***n*=4**。

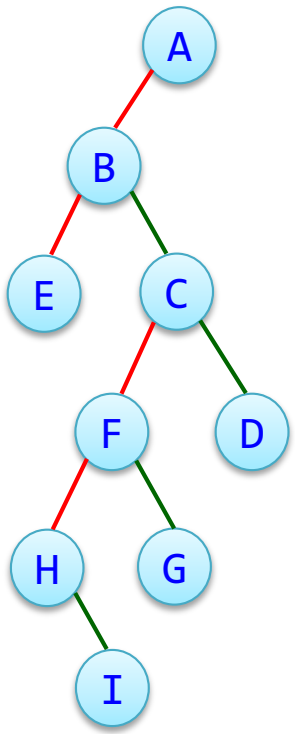
树T中分支结点个数为*n*，则二叉树B中无右孩子的结点个数为*n*+1。



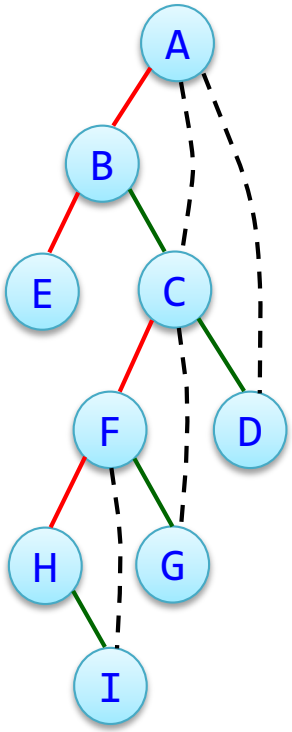
一棵树　　二叉树

T中一个非叶子结点至少有一个孩子结点，其中有一个最右边的孩子结点s
在B中s没有右孩子　⇨　T中*n*个非叶子结点，B中对应*n*个没有右孩子结点
T的根结点对应B的根结点，它一定是没有右孩子结点　⇨　*n*+1

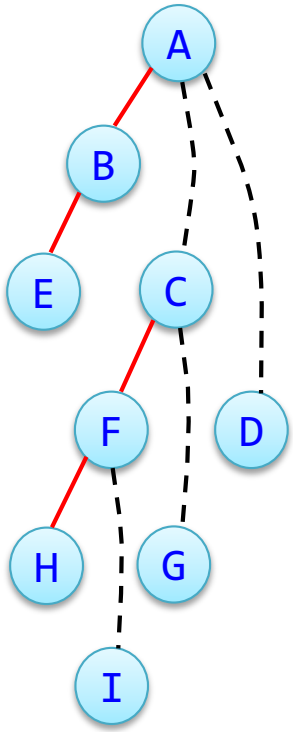
2．一棵由树转换的二叉树还原为树



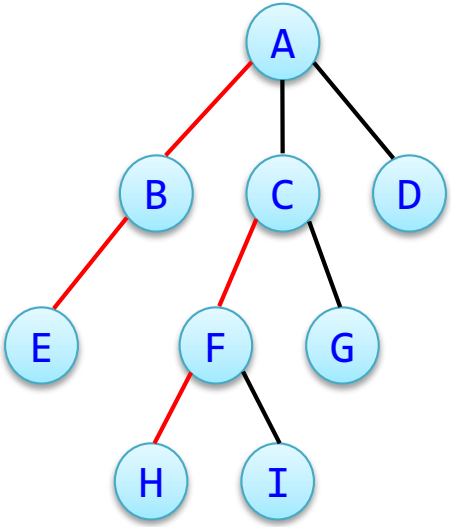
（a）一棵二叉树



（b）加线



（c）抹线



（d）调整

还原的特点

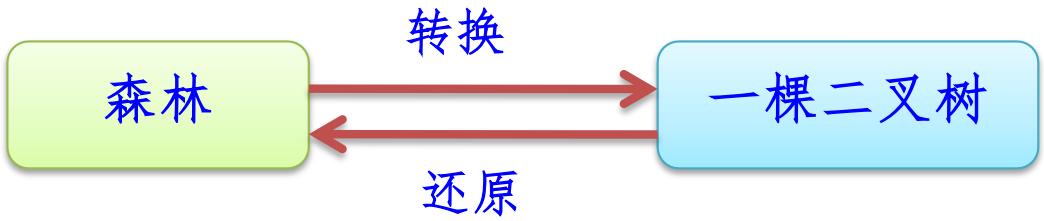
- 根结点不变。
- 左分支不变（左分支为最左孩子），右分支变成兄弟。

2．一棵由树转换的二叉树还原为树

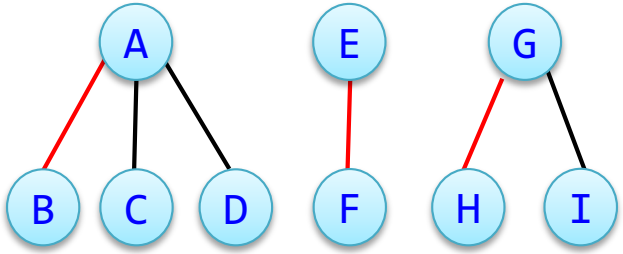
一棵二叉树（由一棵树转换的）可以按照以下规则还原为一棵树：

- 加线：在各结点的双亲与该结点右链上的每个结点之间加一连线，以“双亲—孩子”关系显示表示出来。
- 抹线：抹掉二叉树中所有双亲结点与其右孩子之间的“双亲—右孩子”关系。
- 调整：以二叉树的根结点作为树的根结点，将各结点按层次排列，形成树。

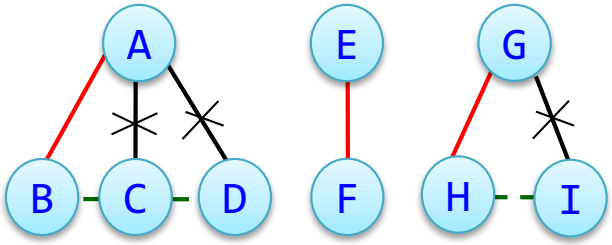
7.8.2 森林与二叉树的转换及还原



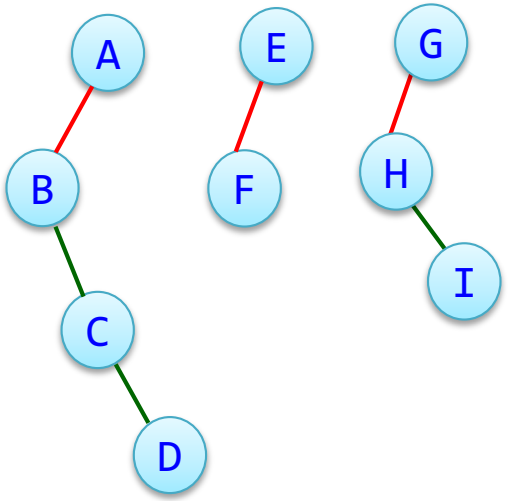
1．森林转换为二叉树



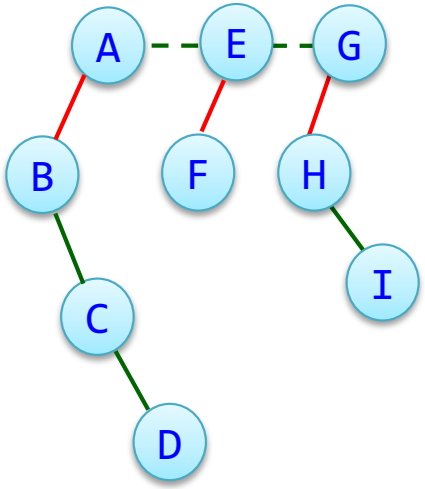
（a）森林



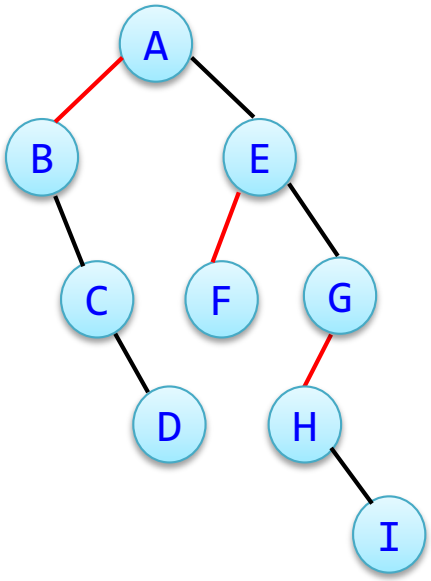
（b）转化为二叉树(1)



（c）转化为二叉树(2)



（d）连线



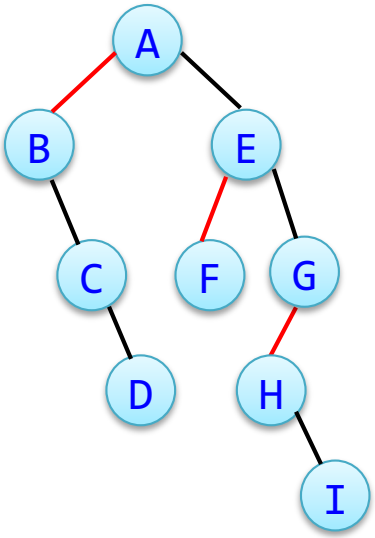
（e）转换成的二叉树

1．森林转换为二叉树

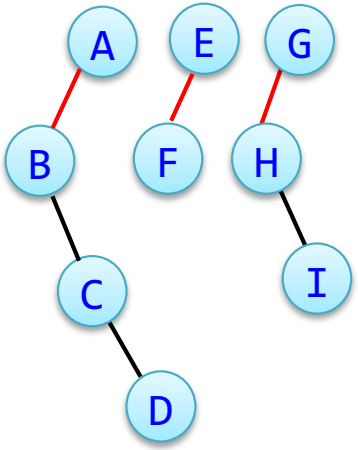
含有两棵或两棵以上树的森林可以按照以下规则转换为二叉树：

- 转换：将森林中的每一棵树转换成二叉树，设转换成的二叉树为bt_1、bt_2、…、bt_m。
- 连接：将各棵转换后的二叉树的根结点相连。
- 调整：以bt_1的根结点作为整个二叉树的根结点，将bt_2的根结点作为bt_1的根结点的右孩子，将bt_3的根结点作为bt_2的根结点的右孩子，…，如此这样得到一棵二叉树，即为该森林转换得到的二叉树。

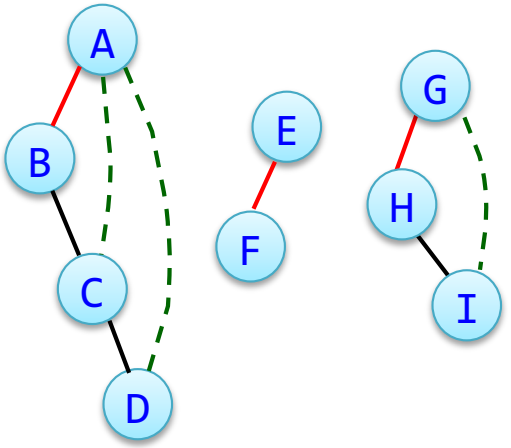
2．二叉树还原为森林



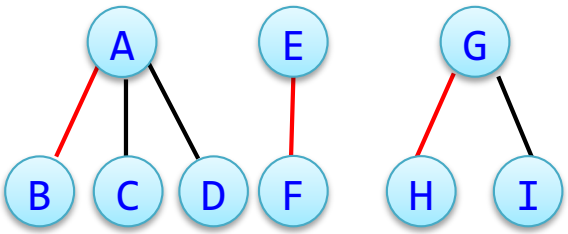
（a）一棵二叉树



（b）抹线



（c）还原为树



（d）还原的森林

2．二叉树还原为森林

当一棵二叉树的根结点有$m-1$个右下孩子，这样还原的森林中有m棵树。这样的二叉树可以按照以下规则还原其相应的森林：

- 抹线：抹掉二叉树根结点右链上所有结点之间的“双亲—右孩子”关系，分成若干个以右链上的结点为根结点的二叉树，设这些二叉树为bt_1、bt_2、…、bt_m。
- 转换：分别将bt_1、bt_2、…、bt_m二叉树各自还原成一棵树。
- 调整：将转换好的树构成森林。

7.9* 并查集

7.9.1 并查集的定义

- 给定*n*个结点的集合，结点编号为1～*n*，再给定一个等价关系，由等价关系产生所有结点的一个划分，每个结点属于一个等价类，所有等价类是不相交的。
- 需要求一个结点所属的等价类，以及合并两个等价类。

求解该问题的基本运算

（1）Init()：初始化。
（2）Find(int *x*)：查找*x*结点所属的等价类。
（3）Union(int *x*, int *y*)：将*x*和*y*所属的两个等价类合并。

上述数据结构称为并查集

经典示例

问题描述：如果已经得到完整的家谱，判断两个人是否亲戚应该是可行的，但如果两个人的最近公共祖先与他们相隔好几代，使得家谱十分庞大，那么检验亲戚关系就十分复杂。在这种情况下，就需要应用并查集。

为了将问题简化，将得到一些亲戚关系的信息，如Marry和Tom是亲戚，Tom和Ben是亲戚，等等。从这些信息中，可以推出Marry和Ben是亲戚。

输入：第一部分以N，M开始。N为问题涉及的人的个数（1≤N≤20000）。这些人的编号为1，2，3，…， N。

下面有M行（1≤M≤1000000），每行有两个数a_i、b_i，表示已知a_i和b_i是亲戚。

第二部分以Q开始。以下Q行有Q个询问（1≤Q≤1000 000），每行为c_i和d_i，表示询问c_i和d_i是否为亲戚。

输出：对于每个询问c_i、d_i，输出一行：若c_i和d_i为亲戚，则输出"Yes"，否则输出"No"。

解决分类问题

输入样例：

```
10 7   //N=10, M=7
2 4
5 7
1 3
8 9
1 2
5 6
2 3
3      //Q=3
3 4
7 10
8 9
```

类似于离散数学中的等价类问题：

给定一个集合U和一个等价关系R，产生具有等价关系的等价类。

采用集合的思路求解

输入关系	分离集合
初始状态	{1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}, {9}, {10}
(2, 4)	{1}, {2, 4}, {3}, {5}, {6}, {7}, {8}, {9}, {10}
(5, 7)	{1}, {2, 4}, {3}, {5, 7}, {6}, {8}, {9}, {10}
(1, 3)	{1, 3}, {2, 4}, {5, 7}, {6}, {8}, {9}, {10}
(8, 9)	{1, 3}, {2, 4}, {5, 7}, {6}, {8, 9}, {10}
(1, 2)	{1, 2, 3, 4}, {5, 7}, {6}, {8, 9}, {10}
(5, 6)	{1, 2, 3, 4}, {5, 6, 7}, {8, 9}, {10}
(2, 3)	{1, 2, 3, 4}, {5, 6, 7}, {8, 9}, {10}

结果集合：{1，2，3，4}，{5，6，7}，{8，9}，{10}

求解：

3 4 ⇨ 3、4在同一个集合中 ⇨ Yes

7 10 ⇨ 7、10不在同一个集合中 ⇨ No

8 9 ⇨ 8、9在同一个集合中 ⇨ Yes

{ {1，2，3，4}，{5，6，7}，{8，9}，{10} }

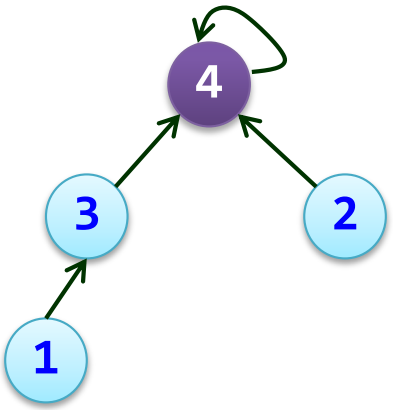
- 并查集的数据结构记录了一组分离的动态集合$S=\{S_1, S_2, \cdots, S_k\}$。
- 每个动态集合S_i（$1 \leqslant i \leqslant k$）通过一个“代表”加以标识，该代表即为所代表的集合中的某个元素。对于集合S_i，选取其中哪个元素作为代表是任意的。

{ {1, 2, 3, 4}, {5, 6, 7}, {8, 9}, {10} }

对于给定的编号为1～n的n个元素，x表示其中的一个元素，设并查集为S，并查集的实现需要支持如下运算：

- **Init(S, n)**：初始化并查集S，即$S=\{S_1, S_2, \cdots, S_n\}$，每个动态集合$S_i$（$1 \leqslant i \leqslant n$）仅仅包含一个编号为$i$的元素，该元素作为集合$S_i$的“代表”。
- **FIND(S, x)**：返回并查集S中x元素所在集合的代表。
- **UNION(S, x, y)**：在并查集S中将x和y两个元素所在的动态集合（例如S_x和S_y）合并为一个新的集合$S_x \cup S_y$。

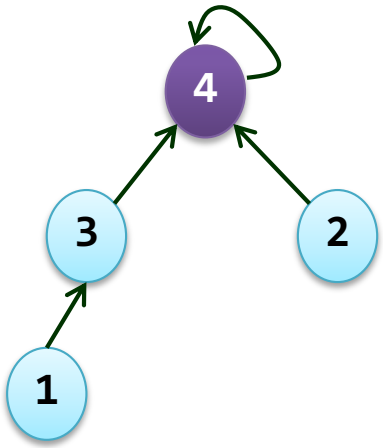
- 用有根树来表示集合，树中的每个结点包含集合的一个成员，每棵树表示一个集合。
- 多个集合形成一个森林，以每棵树的树根作为集合的代表，并且根结点的父结点指向其自身，树上的其他结点都用一个父指针表示它的附属关系。



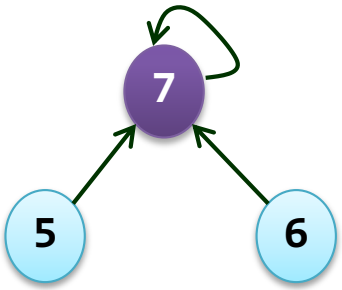
{1，2，3，4}集合

在并查集中，每个分离集合对应的一棵树，称为分离集合树。整个并查集也就是一棵分离集合森林。

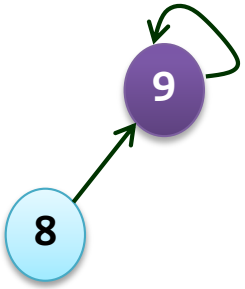
4个集合{1，2，3，4}、{5，6，7}、{8，9}、{10}，分别以4、7、9和10表示对应集合的编号。



{1，2，3，4}集合



{5，6，7}集合

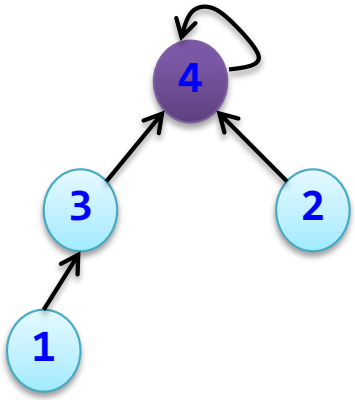


{8，9}集合

{10}集合

几个问题

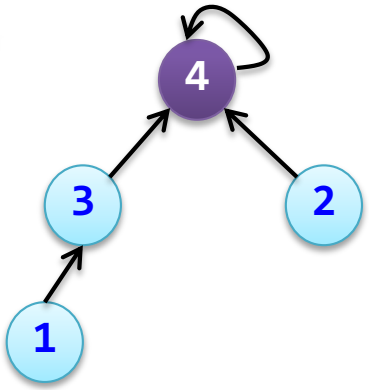
1 查找x



{1，2，3，4}集合

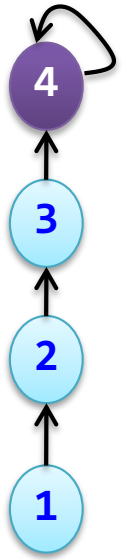
用数组存放：t[x]对应x结点

2 查找x所在的子集



{1，2，3，4}集合

查找1所在的子集合：3次比较



{1，2，3，4}集合

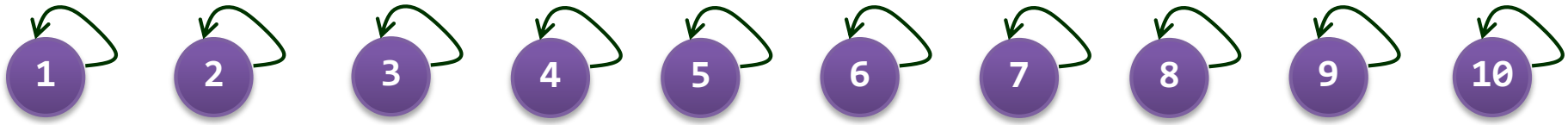
查找1所在的子集合：4次比较

子树高度越小越好

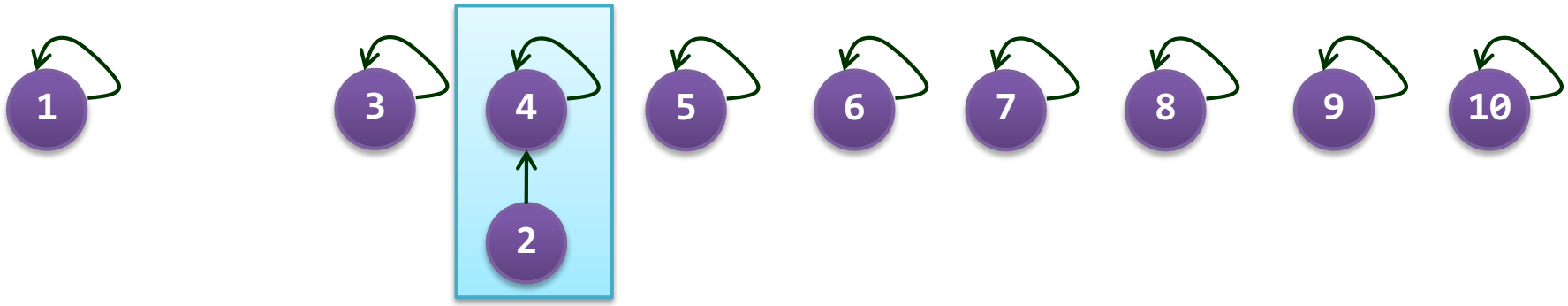
3 合并过程

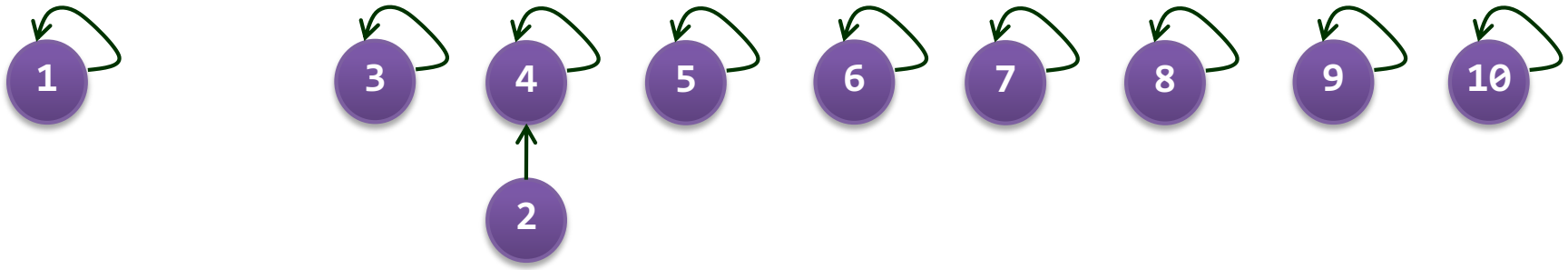
初始状态 {1}，{2}，{3}，{4}，{5}，{6}，{7}，{8}，{9}，{10}

parent



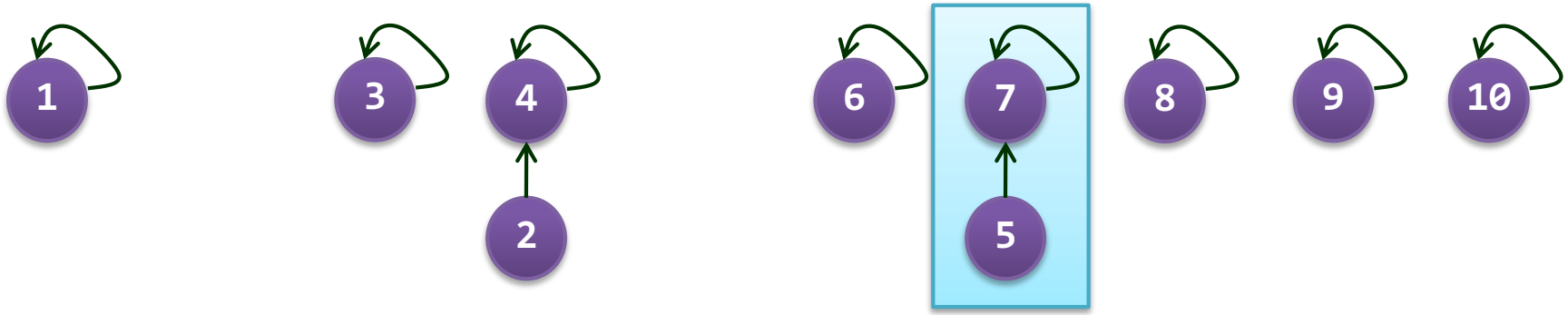
（2，4） {1}，{2，4}，{3}，{5}，{6}，{7}，{8}，{9}，{10}

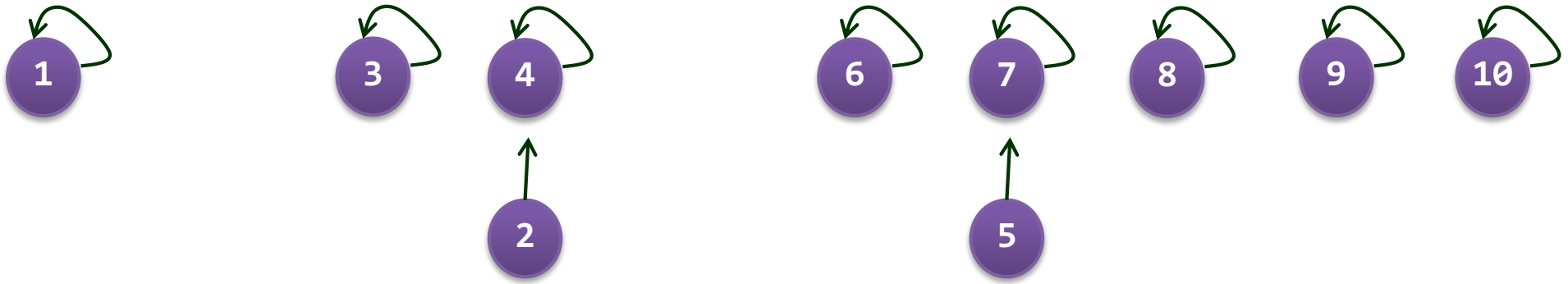




(5 , 7)

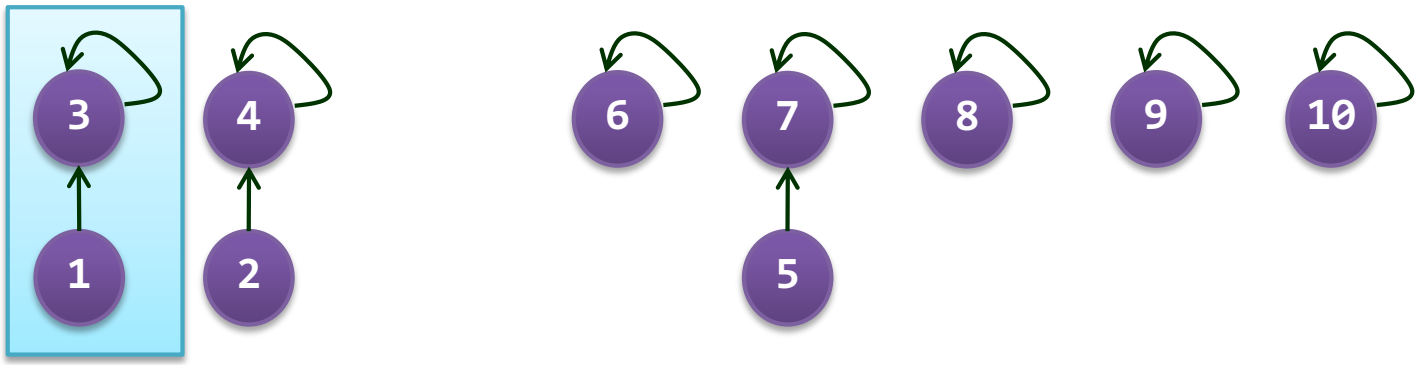
{1} , {2 , 4} , {3} , {5 , 7} , {6} , {8} , {9} , {10}

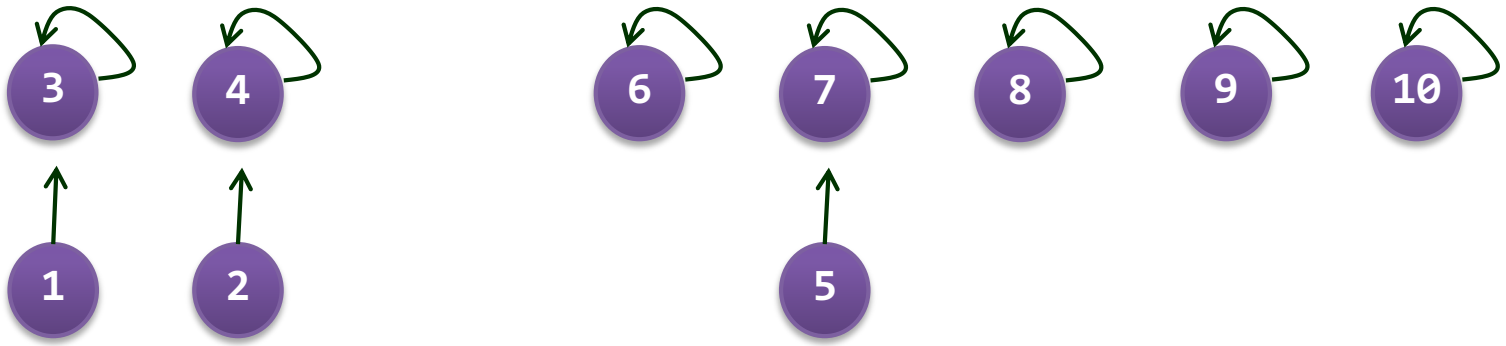




(1 , 3)

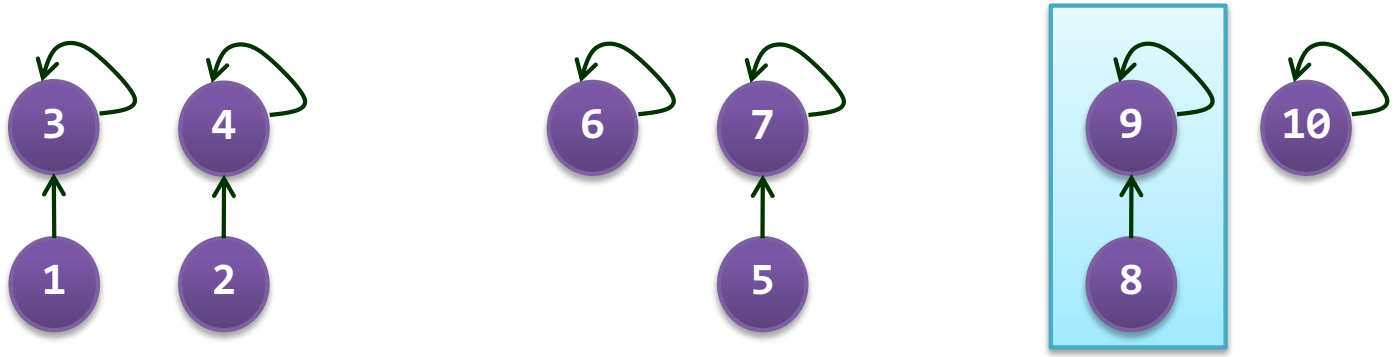
{1 , 3} , {2 , 4} , {5 , 7} , {6} , {8} , {9} , {10}

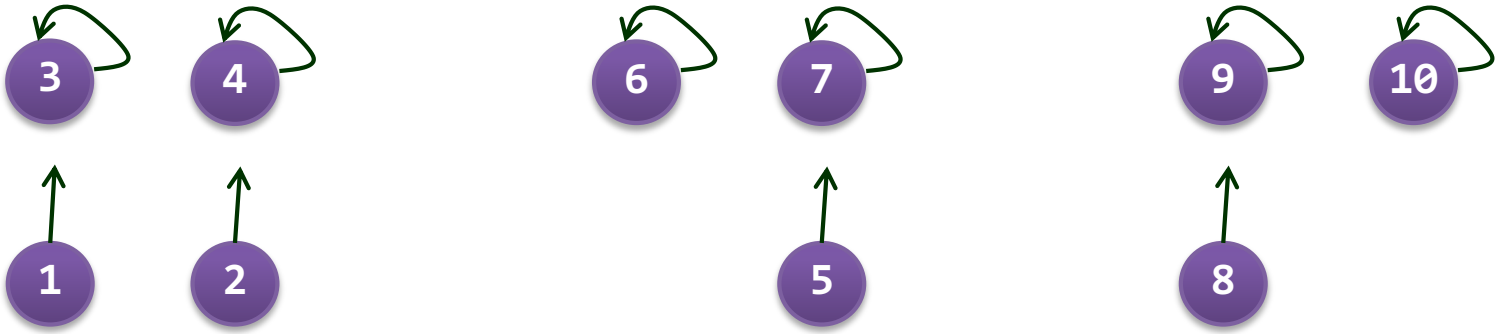




(8 , 9)

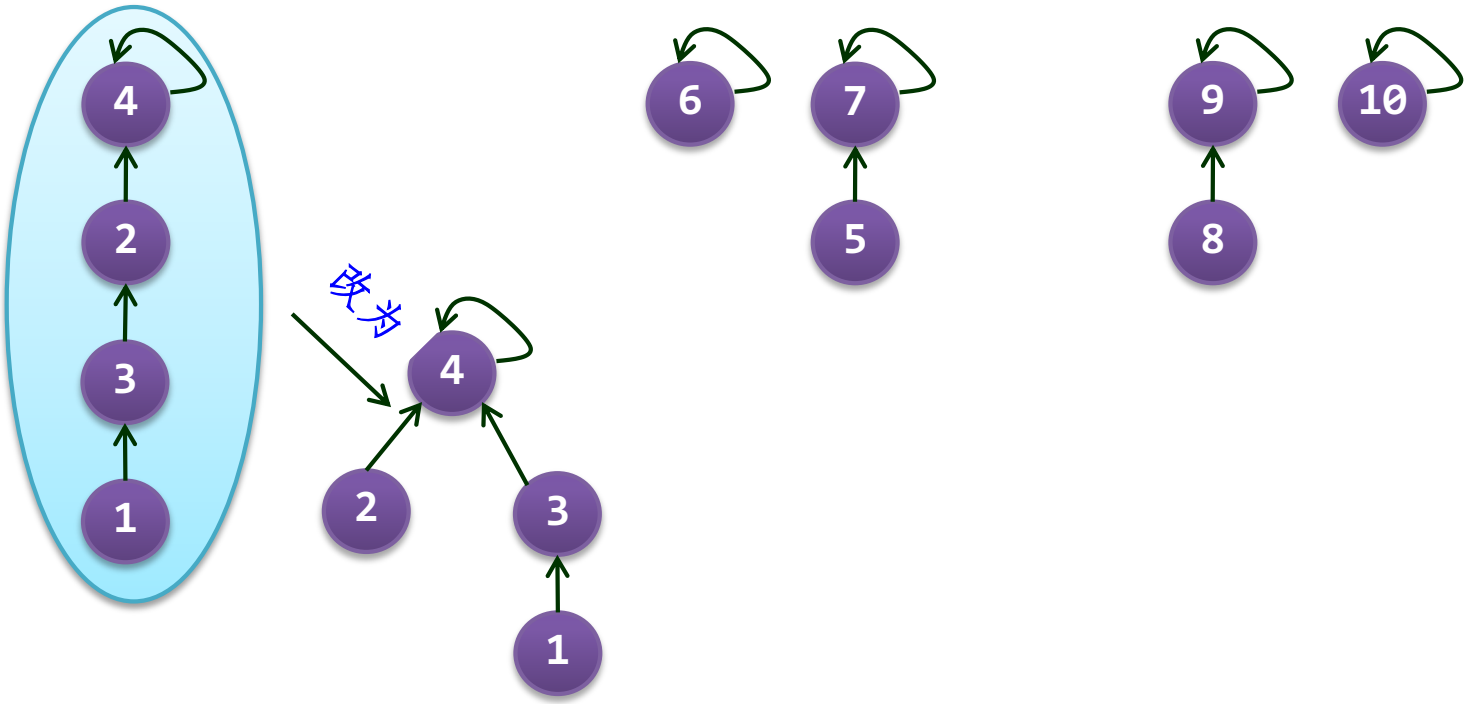
{1 , 3} , {2 , 4} , {5 , 7} , {6} , {8 , 9} , {10}

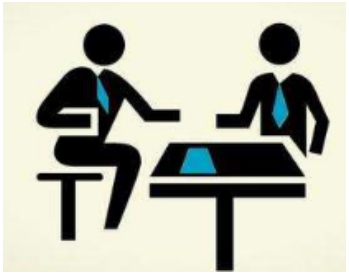




(1 , 2)

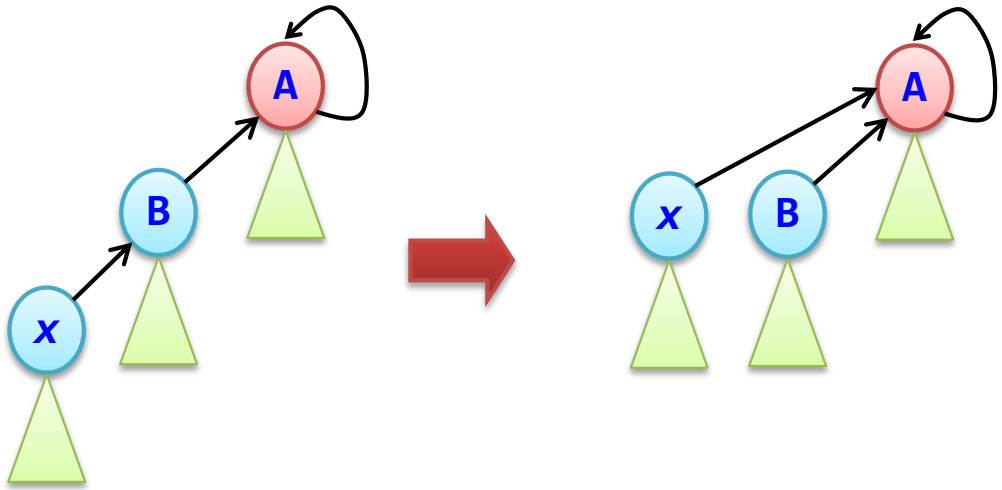
{1 , 2 , 3 , 4} , {5 , 7} , {6} , {8 , 9} , {10}





- 在一棵高度较低的树中查找根结点的编号（即该集合的代表）所花的时间较少，如何保证构造的分离集合树较低呢？
- 两棵分离集合树***A***和***B***高度相同时：任意一个根作为另外一个根的孩子 ⇨ 合并树的高度增加**1**，$h_C=\mathrm{MAX}\{h_A, h_B\}+1$。
- 两棵分离集合树***A***和***B***高度不相同时：将较低的根作为较高的根的孩子 ⇨ 合并树的高度不增加，$h_C=\mathrm{MAX}\{h_A, h_B\}$。

4 查找中的路径压缩



7.9.2 并查集的实现

并查集的基本存储结构（实际上是森林的双亲存储结构）如下：

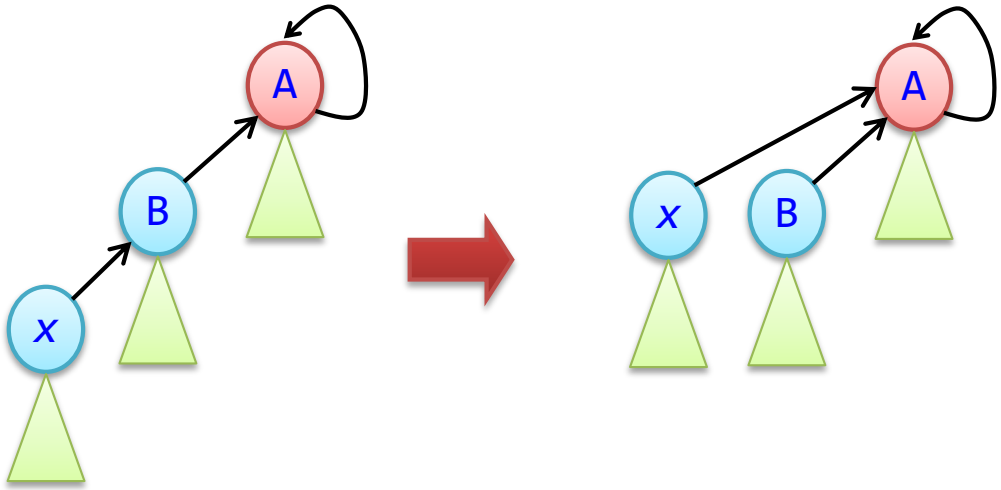
```
int parent[MAXN];                    //并查集存储结构
int rnk[MAXN];                       //存储结点的秩(近似于高度)
```

并查集的基本运算算法

```
void Init()                    //并查集初始化
{  for (int i=1;i<=n;i++)
   {  parent[i]=i;
      rnk[i]=0;
   }
}
```

时间复杂度为O(*n*)。

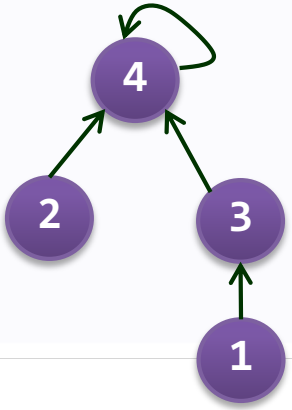
```
int Find(int x)                    //递归算法：并查集中查找x结点的根结点
{  if (x!=parent[x])
     parent[x]=Find(parent[x]); //路径压缩
   return parent[x];
}
```



时间复杂度为$O(\log_2 n)$。

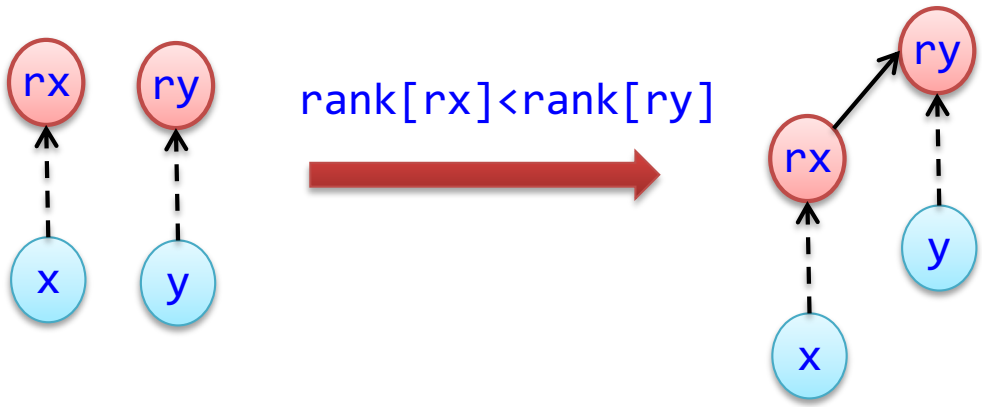
用迭代方式实现

```
int Find(int x)                    //递归算法：并查集中查找x结点的根结点
{  if (x!=parent[x])
     parent[x]=Find(parent[x]); //路径压缩
   return parent[x];
}
```



```
int Find(int x)                    //非递归算法：并查集中查找x结点的根结点
{  int rx=x;
   while (parent[rx]!=rx)          //找到x的根rx
      rx=parent[rx];
   int y=x;
   while (y!=rx)                   //路径压缩
   {  int tmp=parent[y];
      parent[y]=rx;
      y=tmp;
   }
   return rx;                      //返回根
}
```

```
void Union(int x,int y)          //并查集中x和y的两个集合的合并
{  int rx=Find(x);
   int ry=Find(y);
   if (rx==ry)                   //x和y属于同一棵树的情况
      return;
   if (rnk[rx]<rnk[ry])
      parent[rx]=ry;             //rx结点作为ry的孩子
   else
   {  if (rnk[rx]==rnk[ry])      //秩相同，合并后rx的秩增1
         rnk[rx]++;
      parent[ry]=rx;             //ry结点作为rx的孩子
   }
}
```



时间复杂度为$O(\log_2 n)$。

【实战7.5 】HDU1232–畅通工程问题。

问题描述：某省调查城镇交通状况，得到现有城镇道路统计表，表中列出了每条道路直接连通的城镇。省政府“畅通工程”的目标是使全省任何两个城镇间都可以实现交通（但不一定有直接的道路相连，只要互相间接通过道路可达即可）。问最少还需要建设多少条道路？

输入格式：测试输入包含若干测试用例。每个测试用例的第1行给出两个正整数，分别是城镇数目*N*（*N*<1000）和道路数目*M*，随后的*M*行对应*M*条道路，每行给出一对正整数，分别是该条道路直接连通的两个城镇的编号。为简单起见，城镇从1到*N*编号。注意两个城市之间可以有多条道路相通，也就是说：

```
3 3
1 2
1 2
2 1
```

这种输入也是合法的。当*N*为0时，输入结束，该用例不被处理。

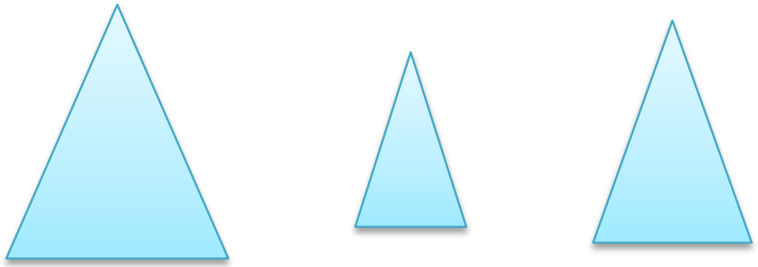
输出格式：对每个测试用例，在一行里输出最少还需要建设的道路数目。
输入样例：

```
4 2
1 3
4 3
3 3
1 2
1 3
2 3
5 2
1 2
3 5
999 0
0
```

输出样例：

```
1
0
2
998
```

- 要使全省任何两个城镇间都实现交通，最少的道路是所有城镇之间都有一条路径，即全部城镇构成一棵树。
- 采用并查集求解，由输入构造并查集，每棵子树中的所有城镇是有路径的，求出其中子树的个数ans，那么最少还需要建设的道路数就是ans-1。



Ans=3 ⇨需要建设的道路数=2

```
#include<iostream>
using namespace std;
const int MAXN=1005;
int parent[MAXN];                    //并查集存储结构
int rnk[MAXN];                       //存储结点的秩
int n;                               //n个城镇
int m;                               //m条道路
void Init()                          //并查集初始化
{  for (int i=1;i<=n;i++)
   {  parent[i]=i;
      rnk[i]=0;
   }
}

int Find(int x)                      //并查集中查找x结点的根结点
{  if (x!=parent[x])
      parent[x]=Find(parent[x]);     //路径压缩
   return parent[x];
}
```

```
void Union(int x,int y)          //并查集中x和y的两个集合的合并
{  int rx=Find(x);
   int ry=Find(y);
   if (rx==ry)                   //x和y属于同一棵树的情况
      return;
   if (rnk[rx]<rnk[ry])
      parent[rx]=ry;             //rx结点作为ry的孩子
   else
   {  if (rnk[rx]==rnk[ry])      //秩相同，合并后rx的秩增1
        rnk[rx]++;
      parent[ry]=rx;             //ry结点作为rx的孩子
   }
}
```

```
int main()
{  while(~scanf("%d",&n))
   {  if(n==0) break;
      scanf("%d",&m);
      Init();                          //初始化
      for (int i=1;i<=m;i++)           //输入m条边
      {  int a,b;
         scanf("%d%d",&a,&b);
         Union(a,b);
      }
      int ans=0;
      for (int i=1;i<=n;i++)           //求子树个数ans
          if (parent[i]==i)
             ans++;
      printf("%d\n",ans-1);            //结果为ans-1
   }
}
```